# DATA FUSION SCENARIO BASED Q&A

BY - SHUBHAM WADEKAR

Copyright © Shubham Wadekar. All rights reserved.

This material is for personal use only. No part may be copied, shared, resold, or published without written permission. Unauthorized distribution is strictly prohibited and may result in action.

For permissions or licensing, contact: [shubham.p.wadekar@gmail.com]

Disclaimer: This content is for educational purposes. Accuracy is attempted but not guaranteed.

No job outcome is promised.

© Shubham Wa

**1. Your company is evaluating ETL tools - how would you justify using Cloud Data Fusion over building custom Spark jobs on Dataproc?**

I would start by saying both approaches are valid, but the choice depends on team skills, business priorities, and operational trade-offs. With Dataproc and Spark, you get full flexibility and control - you can code custom transformations, optimize performance, and design pipelines your way. But that comes with overhead: engineers must write and maintain Spark code, manage schema evolution, handle orchestration, and troubleshoot failures. For most day-to-day ingestion and transformation needs, that becomes heavy and costly over time.

Cloud Data Fusion, on the other hand, provides a managed, low-code environment built on top of CDAP. Instead of coding every transformation, you can design pipelines visually using pre-built connectors and transformations. This accelerates development, reduces human error, and makes pipelines easier for analysts and less technical staff to understand. For common patterns - like ingesting from GCS to BigQuery, applying lookups, joins, or format conversions - you can get pipelines running in hours instead of weeks of Spark coding.

Another key point is operations. With custom Spark jobs, your team needs to maintain versioned code, deployment pipelines, Dataproc clusters, and monitoring for jobs. With Data Fusion, much of that is handled for you - pipelines are version-controlled within the tool, you get lineage and monitoring dashboards out of the box, and jobs run on ephemeral Dataproc clusters without you worrying about cluster lifecycle. That reduces operational burden significantly.

Governance and lineage are another advantage. Data Fusion integrates natively with Data Catalog to automatically track datasets, schema changes, and lineage. If compliance or auditability is important, this is a big selling point compared to homegrown Spark jobs where lineage has to be built manually.

So in an interview, I'd conclude like this: if you have a highly skilled Spark engineering team and unique, complex transformations, custom Dataproc jobs may be the right fit. But if the goal is to empower more teams, reduce time-to-market, and lower operational overhead, Cloud Data Fusion is the smarter choice because it balances flexibility with managed simplicity.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

**2. A manager asks why Cloud Data Fusion (managed) is better than open-source CDAP. How would you explain it?**

I'd explain that CDAP is the open-source framework that powers Data Fusion, so technically you could self-host CDAP and get similar functionality. But running open-source CDAP in production means you're responsible for everything - cluster management, upgrades, security patches, scaling, monitoring, integration with GCP services, and troubleshooting issues without guaranteed support. That's a big ongoing investment in time and expertise.

Cloud Data Fusion, being the managed service, takes away all that overhead. It's fully integrated into GCP - meaning authentication is through IAM, lineage flows into Data Catalog, logs and metrics flow into Cloud Logging and Monitoring, and jobs run on managed Dataproc clusters automatically. You don't have to maintain the infrastructure, apply patches, or worry about scaling up/down. Google handles all of that.

Another difference is support. With open-source CDAP, if something breaks, your engineers are on their own to debug or rely on community support. With Cloud Data Fusion, you get enterprise-grade support through Google Cloud, which is critical for production workloads. Managers care about accountability and SLAs, and managed Data Fusion provides that out of the box.

Also, managed Data Fusion evolves faster with pre-built plugins and connectors for GCS, BigQuery, Pub/Sub, Salesforce, and other services. These connectors are tested and maintained by Google, reducing integration risk. In open-source CDAP you'd need to build, test, and maintain those connectors yourself.

So the way I'd put it to a manager: CDAP gives you raw power but also raw responsibility. Cloud Data Fusion gives you the same foundation but with enterprise-grade reliability, native GCP integration, support, and reduced operational overhead. For a business that wants to focus on delivering data value rather than managing infrastructure, Cloud Data Fusion is the better long-term choice.

**3. You're asked to design a pipeline that shows lineage for auditing. How would you implement this in Data Fusion?**

Start by treating lineage as first-class metadata - instrument every pipeline to emit provenance at each logical step. In Data Fusion enable the built-in lineage features (the Metadata and Lineage integration) so every pipeline run automatically captures source/destination dataset identifiers, transformation nodes, and dataset-level lineage. Use canonical dataset URIs (gs://..., bq://project.dataset.table) as the stable keys so auditors can find assets unambiguously. Version every pipeline and capture the pipeline_id + pipeline_version in the run metadata.

Augment automatic lineage with explicit provenance columns and manifests. Have upstream jobs write a manifest (ingest_id, source_path, generationId, producer, timestamp) to a metadata table (BigQuery). Each transform should carry forward the ingest_id so downstream tables contain audit columns (source_file, ingest_time, ingest_id). This makes row-level tracing possible when auditors ask "which file produced this record" without having to reverse-engineer pipelines.

Integrate Data Fusion lineage with GCP services for storage and search. Push pipeline metadata and lineage to Data Catalog (Data Fusion can publish metadata) and export run-level details to a dedicated BigQuery lineage_audit table for fast querying and reporting. Keep copies of pipeline run logs in Cloud Logging and create log-based metrics for anomalies (unexpected source changes, schema drift). The combination of Data Catalog + BigQuery audit table gives both GUI discovery and SQL-queryable audit trails.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

Add immutable artifacts and retention policies: store pipeline artifacts (templates), transformation configs, and validation reports in a versioned storage location (GCS) and attach checksums. Keep lineage records immutable for the audit retention period and protect them with IAM so only auditors and compliance roles can modify. Provide an admin replay/forensics tool (small Cloud Run app or notebook) that, given an ingest_id or table partition, traces upstream lineage and returns the chain of files/pipelines/jobs.

Operationalize with SLA guards and alerts: validate lineage completeness on each run (pipeline run must produce lineage entries and manifest entries) and alert on missing or partial lineage. Provide sample queries/report templates for auditors (e.g., "files → pipelines → table partitions for date X") and document runbook steps for forensic replay. This combination - Data Fusion lineage, Data Catalog, manifest fields, and a queryable lineage_audit store - gives auditors both GUI and SQL access to prove provenance.

**4. A pipeline is running slowly - how would you decide whether to use Direct, Spark, or Dataproc runtime?**

Make the decision from measurable signals: input size, shuffle/joins, memory/CPU profile, latency SLO, and dependencies. Start by profiling: run a representative job and collect metrics (CPU, memory, disk spill, shuffle bytes, task durations). If the job handles small datasets (MBs–low GBs), few shuffles, and needs low start-up latency, Direct runtime (local execution) is often fastest and cheapest - it avoids cluster startup and is simpler to operate.

If your pipeline performs distributed transforms (large joins, aggregations, heavy shuffles) and you need parallelism across many cores, prefer Spark runtime. Spark gives a distributed execution engine with in-memory shuffle and caching that handles medium-to-large datasets well. Choose the Data Fusion Spark runtime when you want faster distributed execution but still want the managed convenience of ephemeral execution without managing clusters yourself. Monitor for expensive shuffle/spill metrics - if tasks are spilling to disk or shuffles are huge, Spark with tuned memory and shuffle configs will help.

Pick Dataproc runtime (managed Spark on Dataproc) when you need full control over cluster sizing, custom libraries, or advanced tuning - for example, custom jars, native libraries, GPU nodes, or long-running clusters that amortize startup cost. Dataproc is the choice when the workload is very large (many TBs), needs specific VM types, or benefits from persistent clusters with autoscaling policies and custom initialization actions. It's also appropriate when you need fine-grain control for performance tuning (YARN configs, executor memory, shuffle service).

Also weigh operational trade-offs: Direct is simplest but limited in scale; Spark runtime is a balanced managed option for most production ETL; Dataproc gives maximum control and is best for extreme scale or custom needs but increases operational burden. Run a short-cost/latency experiment: execute the same job on each runtime with production-size sample and compare wall-clock time, cost, and resource utilization. Use that empirical data plus SLA requirements to pick the runtime.

Finally, iterate: select the simplest runtime that meets SLAs, then tune. For Spark/Dataproc adjust executor counts, cores, memory, shuffle partitions, and input partitioning; for Direct optimize transforms and push heavy work downstream. Document your performance tests and keep a monitoring dashboard so you can detect regressions and re-evaluate runtime choice as data volumes or SLAs change.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

**5. Your project already uses BigQuery and Pub/Sub heavily. How would you integrate them efficiently with Data Fusion?**

I'd treat BigQuery and Pub/Sub as first-class sources/sinks and use Data Fusion's native plugins so integration is configuration, not glue code. For Pub/Sub ingest, use the Pub/Sub source plugin (or a lightweight Cloud Function that forwards messages to a topic Data Fusion reads from) so streaming events enter the pipeline reliably. For BigQuery output use the BigQuery sink plugin and choose the sink mode (staging+load or streaming) based on latency and volume needs. Using native connectors keeps authentication, retries, and schema mapping consistent and reduces custom maintenance.

Design for clear routing and staging: for streaming or high-volume messages, consider writing validated/normalized batches to GCS (ndjson/Avro/Parquet) and then use BigQuery load jobs; this is cheaper and more robust than one-row streaming when throughput is large. For low-latency needs, the BigQuery sink or Storage Write API-backed connector is fine, but add batching in the pipeline to reduce per-row overhead. Keep an intermediate staging layer (GCS or staging tables) so you can replay, audit, and re-run loads without reprocessing Pub/Sub history.

Make schema and lineage explicit. Store schema artifacts in a schema registry (or in Data Catalog) and have the pipeline validate incoming Pub/Sub messages against the expected schema early. Enable Data Fusion's metadata/lineage features so each pipeline run records source topic → transform → BigQuery table lineage into Data Catalog. Also add ingest_id or message_id as provenance columns during transformations so row-level tracing from BigQuery back to Pub/Sub is possible for debugging and audits.

Operationally, harden the pipeline with DLQs, retries, and monitoring. Configure Pub/Sub subscriptions with DLQs for poison messages and let Data Fusion emit failed-records to a rejects GCS prefix or BigQuery rejects table. Add log-based metrics and dashboards for Pub/Sub backlog, Data Fusion pipeline latency, BigQuery load errors and quotas. Use job labels and correlation ids so logs, billing and error alerts can be stitched end-to-end.

Finally, secure and optimize for cost. Use least-privilege service accounts for Data Fusion to read Pub/Sub and write BigQuery/GCS, enable BigQuery partitioning/clustering to reduce query cost, and choose load vs streaming based on cost/latency tradeoffs. If you need custom heavy transforms, run Data Fusion on Spark/Dataproc runtime so you get scalable execution while still using the same connectors - this minimizes custom code and speeds delivery.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

**6. You're ingesting daily sales files from GCS - would you choose batch or streaming? Why?**

For daily sales files I'd choose batch in almost all cases. The files arrive as discrete artifacts representing a day's worth of data, business users expect correctness and completeness per-file, and latency requirements are typically coarse (daily or hourly). Batch (GCS → load job → staging → MERGE) gives atomicity, lower cost per GB, and simpler error handling - you can validate the whole file, run schema checks, and do an atomic MERGE into the production partition. That makes audits and replays straightforward.

Batch also lets you use cheaper GCS→BigQuery load jobs and take advantage of partitioned tables and transactional semantics. Large daily files are more efficiently processed with bulk loads than with millions of per-row streaming calls - load jobs avoid per-row costs, support schema options (ignoreUnknownValues, null handling), and are easy to retry or re-run when something fails. Manifest-driven ingestion (record file generationId and status) ensures idempotency and easy replay for operators.

Only choose streaming if business requires sub-minute freshness or real-time dashboards for sales (fraud detection or capacity decisions). In that case use a hybrid approach: stream critical events (e.g., high-value transactions) via Pub/Sub → streaming pipeline for immediate alerts, while still doing the full daily batch for reconciliation and the canonical dataset. This gives the best of both worlds without complicating the canonical data model.

Operationally, batch simplifies SLA and cost management - you can schedule windows, size resources for predictable times, and keep monitoring focused on per-file success/failure. It also makes downstream consumers (reports, Looker) simpler because they can rely on daily, partitioned snapshots rather than continuous partial updates. Provide a small near-real-time feed if needed, but keep the daily batch as the authoritative source.

In short: choose batch for daily GCS sales files for simplicity, cost-efficiency, and correctness; use streaming only as a complementary fast-path for specific low-latency business needs.

**7. Your company wants near-real-time analytics for IoT sensor data. How would you build this in Data Fusion?**

Start with the ingestion layer: push sensor events into Pub/Sub as the universal, scalable ingress. Use Data Fusion's Pub/Sub source (streaming runtime) to consume messages, or use a tiny Cloud Function/Cloud Run forwarder if you need protocol translation. Keep the message schema compact (timestamp, device_id, metric, quality flags) and include an event_time so downstream can do event-time processing.

In the Data Fusion pipeline, do minimal, fast enrichment and validation: parse JSON, drop malformed fields, canonicalize timestamps, attach a correlation id and device metadata (cached lookup from Memorystore or a small BigQuery dimension table). The goal is to keep the streaming pipeline low-latency - only light transforms in-line, heavy joins or model scoring should be offloaded. Use Data Fusion streaming transforms or route messages to a downstream Dataflow job when you need advanced event-time/window semantics.

For analytics sinks, write clean events to a low-latency store like BigQuery (streaming/batched with Storage Write API) for analytics and also to a time-series store (Bigtable or Influx/TSDB) for fast point-lookup and dashboarding. Use partitioned, clustered BigQuery tables optimized by device_id and event_date so queries are efficient. If you need materialized aggregates for dashboards, have a downstream micro-batcher (Cloud Run or Dataflow) that writes periodic aggregates to a summary table that Looker/Tableau can query.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

Handle scaling and backpressure by separating concerns: Data Fusion handles ingestion/validation and publishes to intermediate topics or staging buckets when load spikes; dedicated batch or streaming workers do aggregation and heavy enrichment. Add DLQs for poison messages and a rejects prefix in GCS for troubleshooting. Monitor Pub/Sub backlog, pipeline latency, and write failures and set autoscaling limits or fallbacks to avoid cascading failures.

Finally, operationalize: provide per-device SLA dashboards (latency, last-seen), retention/lifecycle for raw events in GCS, and a replay mechanism so historical windows can be recomputed. Secure the pipeline with least-privilege service accounts and use correlation ids for tracing. This design gives near-real-time visibility while keeping the pipeline robust and maintainable.

**8. Events arrive late and out of order in Pub/Sub - how would you handle this in a Data Fusion streaming pipeline?**

First, treat event_time as authoritative - never rely on ingest time for analytics. Ensure each message carries the original event timestamp and preserve it through the pipeline. In Data Fusion (or better, in an underlying Dataflow/Beam job) use event-time windowing rather than processing-time windows so your aggregates reflect when the event actually happened. Configure appropriate window sizes and slide intervals to match business needs.

Second, use watermarks and allowed lateness. Define a watermark that represents how far behind you're willing to accept late events (for example, 5–15 minutes). Configure the windowing logic to accept late data up to that allowed lateness and to update aggregates (accumulating/merging mode) when late events arrive. For events later than the allowed lateness, route them to a "late-events" sink (GCS/BigQuery rejects) for manual review or to a compensating recompute flow.

Deduplication is important when out-of-order delivery or retries occur. Include a unique event_id (producer-assigned or derived from device+seq+timestamp) and dedupe in the pipeline using a short-lived state store (in-memory cache, Memorystore, or Beam state) or by writing to a staging table with event_id uniqueness enforced and MERGE semantics. This prevents double-counting when late duplicates arrive.

Recognize that some correctness guarantees are easier in a streaming engine with strong windowing support (Beam/Dataflow) than in a purely visual streaming pipeline. If your Data Fusion streaming runtime can't express the required event-time/watermark/dedup semantics, have Data Fusion hand off to a Dataflow job for the windowed aggregations and then consume the post-processed results back into Data Fusion sinks. Also provide a periodic batch recompute (nightly) that reconciles any late arrivals that missed the allowed-lateness window.

Finally, instrument and operationalize: track late-event rates, watermark lag, and number of recomputes; alert when late-event volume grows or when dedupe rates rise (indicating producer issues). Keep a replay path so you can reprocess late files and ensure your downstream aggregates can be corrected deterministically. With event-time processing, watermarks, dedupe keys, and a reconciliation path you'll handle out-of-order and late events robustly while keeping analytics accurate.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

**9. A client asks why you wouldn't always use streaming instead of batch. How would you answer?**

Streaming is great for low-latency needs, but it's not a one-size-fits-all solution. The first point I'd make is cost and efficiency: streaming systems typically pay per-operation or per-message and keep resources warm for long periods. For very large volumes where you only need hourly/daily views, batch (bulk loads) is much cheaper and often faster overall because you can amortize startup and I/O costs and use efficient bulk load paths (GCS → BigQuery load jobs) rather than millions of small writes.

Second, correctness and simplicity matter. Batch gives you an easier mental model for atomicity, deterministic reprocesses, and reconciliation. If business needs are about complete-day accuracy, reconciliation, or complex joins across the entire day, it's simpler and less error-prone to run a single batch job that validates, transforms and merges the full dataset. Streaming requires building robust windowing, watermarking, dedupe, and late-arrival handling which increases complexity and operational burden.

Third, operational overhead and developer velocity. Streaming pipelines need more sophisticated monitoring (watermarks, backlog, replay paths), careful handling of schema evolution, and harder debugging for out-of-order or late data. Teams without mature streaming experience often spend more time on reliability engineering. Batch pipelines are easier to test, replay, and reason about, and they fit well with many compliance/audit requirements because each file or partition is an immutable unit of work.

Fourth, latency vs value trade-off. Ask: does the business actually gain value from sub-minute freshness? If the business KPI tolerates minutes or hours of lag, the additional cost and complexity of streaming often aren't justified. A hybrid approach is usually best: stream a small subset of critical events for near-real-time signals (alerts, dashboards), and keep the batch as the canonical, auditable source for reconciliation and reporting.

Finally, technical constraints matter: some sinks and analytics (expensive global joins, heavy windowing, model training) are more naturally and economically expressed in batch. So I'd recommend choosing streaming only when latency is a hard requirement and you're prepared for the extra operational and engineering cost; otherwise prefer batch or a hybrid pattern.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

**10. Your team wants micro-batch processing for a clickstream dataset. How would you design this?**

I'd design a micro-batch pipeline that balances latency, cost, and correctness by using a reliable ingestion layer, short fixed windows for batching, and idempotent sinks. Start with Pub/Sub as the ingestion bus for click events from clients (web/mobile). Each event should carry an event_time, device/session id, and a small sequence or event id for dedupe. Pub/Sub gives scale and buffering while keeping producers simple.

For the micro-batch worker, use a managed streaming/batch engine that supports windowing and exactly-once semantics for practical dedupe - Dataflow (Beam) or a short-interval Spark job on Dataproc/Dataproc Serverless. Configure the job to create fixed micro-batches (for example 1–5 minute tumbling windows) where events are grouped, deduped by event_id, and aggregated (sessionization, pageviews, clicks per user). Use watermarks and a small allowed-lateness window to accept slightly late events and produce correct aggregates.

Write outputs to two sinks: a low-latency analytics table (BigQuery partitioned by event_date and clustered by user/session) for near-real-time dashboards, and an archival raw-events store in GCS (partitioned by date/hour) for replay and historical recompute. Use staging tables or write Parquet/Avro files and trigger load jobs to BigQuery to keep costs down and ensure atomic loads for each micro-batch. Include ingest_id and window boundaries in metadata so each batch is traceable and replayable.

Operational controls are key: implement idempotency (manifest table per-batch with status PENDING→PROCESSING→SUCCESS), DLQs for poison messages, and metrics for batch latency, throughput, and late-event rates. Add alerting for growing Pub/Sub backlog or slow micro-batch completion. Provide a replay admin path to re-run specific windows using stored raw files if bugs or model changes require backfill.

Finally, tune based on SLAs and volume: choose batch size for acceptable freshness vs cost (smaller windows = lower freshness but higher overhead), optimize partitioning and clustering in BigQuery for query patterns, and version aggregation logic so you can safely roll forward/back. This micro-batch pattern gives near-real-time insights with simpler correctness and replayability than fully streaming, which is often ideal for clickstream analytics.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

**11. You built a pipeline in Studio - how would you promote it to production?**

Treat promotion as a controlled CI/CD flow, not a manual copy. First, store the pipeline definition in version control (export the Studio pipeline as JSON/YAML or use Studio's Git integration) so every change is tracked and code-reviewed. Keep environment-specific values (project IDs, buckets, dataset names, credentials) out of the pipeline file and replace them with runtime parameters or a config file so the same artifact can run in dev/test/prod without edits.

Add automated tests and a staging run. Before promoting, run unit-style tests (small sample inputs → expected outputs) and an integration smoke test in a staging environment that mirrors prod (same plugin versions, similar data volume if feasible). Capture metrics (runtime, row counts, error rates) and compare against baselines so you know the staging run behaved correctly. Only allow promotion after automated tests and a quick manual sanity check.

Automate deployment. Use a CI job (Cloud Build/GitHub Actions) that, on merge to the main/prod branch, exports the pipeline artifact and invokes the Data Fusion REST API or uses the CLI to create/update a pipeline in the production instance (or pushes a flex template if the heavy lift is in Dataflow). Tag the artifact with the Git commit and pipeline version, and record that version in a deployment manifest table for auditability. Keep an approval gate in CI if your org needs manual sign-off.

Implement runtime controls and monitoring. When the pipeline is created in prod, supply production parameters via a secrets-backed config (Secret Manager) and set up alerting (log-based metrics, pipeline health) and lineage (Data Catalog) hooks. During the first few runs, run the job in "dry-run" or limited-scope mode (process a small partition) if possible, then increase to full production. Capture run ids and update the manifest with outcome, duration, and any anomalies.

Have rollback and replay plans. If a promoted pipeline behaves badly, the CI system should be able to redeploy the previous pipeline version quickly (artifact stored in version control). Also provide a replay mechanism (re-run pipeline for a given ingest_id or date) using stored raw inputs and manifests so you can rebuild downstream state without manual rework. Document the promotion checklist and keep a short runbook for on-call - auditors and interviewers like seeing traceability, automation, and safe rollbacks.

**12. A pipeline needs to run daily at midnight. How would you schedule it?**

Pick a scheduler with clear observability and timezone handling. The simplest robust option is Cloud Scheduler to trigger the pipeline daily at 00:00 in the business timezone (store the timezone explicitly, e.g., Asia/Kolkata). Cloud Scheduler can invoke an HTTP endpoint (a small Cloud Function or Cloud Run service) which in turn calls the pipeline start API (Data Fusion REST, Composer DAG trigger, or your pipeline's run endpoint) and passes any runtime parameters. This decoupling gives you retry control, auth isolation, and richer logging.

If you already use Composer/Airflow, create a DAG with a schedule_interval='0 0 * * *' and explicit timezone set to the business zone and have the DAG trigger the Data Fusion pipeline (via the REST hook or Pub/Sub). Composer gives you built-in retries, UI history, SLA monitoring, and backfill capabilities - useful if downstream replay or backfills are common. Use Composer if you also orchestrate other daily tasks.

Make the scheduling idempotent and manifest-driven: when the scheduler triggers the run, the starter service should write a manifest row (batch_date, ingest_id, status=PENDING) and check for an existing SUCCESS entry for the same batch_date to avoid double-runs from accidental re-triggers. The starter should pass batch_date as a parameter to the pipeline so the job is parameterized and repeatable. Log the run_id returned by the pipeline and update the manifest with run status on completion.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

Add retries, dead-letter handling and alerting. Configure the starter (Cloud Function) with bounded retries and exponential backoff for transient errors; if repeated attempts fail, mark the manifest FAILED and push an alert (Slack/PagerDuty) with the correlation id and logs. Also monitor for missed schedules (Cloud Scheduler provides metrics) and set an alert if a scheduled run doesn't start within an expected window.

Finally, document timezone and maintenance windows. Explicitly record which timezone the cron uses in pipeline docs and manifest rows, and plan for daylight-savings or DST-free timezones if applicable. Provide an admin endpoint to manually trigger or re-run a midnight job for a specific date (using the same parameterized path) so operators can recover quickly from failures - interviewers expect scheduling + idempotency + observability, not just a cron.

**13. A pipeline fails in production. How would you debug and monitor logs?**

First, don't start by guessing - gather the right identifiers. Find the pipeline run id, correlation_id/ingest_id, and timestamps from your manifest or Data Fusion UI. Those IDs let you tie together pipeline runs, Cloud Logging entries, BigQuery job ids, Pub/Sub message_ids and any downstream job ids. Having those upfront makes searches precise and avoids noise.

Next, use centralized structured logs and traces. Open Cloud Logging and filter by the correlation_id and the pipeline run id; inspect the sequence of events (validation → transform → load). Look for the first ERROR/EXCEPTION or for non-200 responses from downstream APIs. If you enabled tracing or OpenTelemetry, follow the trace to see latency spikes or retries. Also check Data Fusion's pipeline run details and task-level logs - the UI will show which plugin or transform failed.

Check downstream systems and manifests - a BigQuery load error or Dataflow worker failure is often the real cause. Pull the exact BigQuery job id from the logs and inspect the job errors in BigQuery UI; for Dataflow look at job logs and worker autoscaling events. Don't forget infra signals: Pub/Sub backlog, Dataproc / Dataflow worker preemptions, or service quota errors often cause intermittent failures. Correlate timestamps across systems.

After you've identified the root cause, capture a reproducible failure case: either a small sample payload that triggers the bug or the exact failing file. Re-run the pipeline in a staging environment with the same parameters and enable verbose/debug logging. Fix the code or parameter, then re-run the failed run (replay using the manifest row) and verify success. Record the root cause and remediation in the incident ticket.

Finally, close the loop operationally: add a log-based metric or alert for the specific error condition so the next occurrence triggers immediate notification, and consider guardrails (input validation, schema checks, DLQ, retry limits) to prevent recurrence. Update the runbook with the steps you followed and add a post-mortem with timeline, fix, and preventive actions so the team can avoid the same issue later.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

**14. A pipeline must run only after another pipeline finishes successfully. How would you orchestrate this?**

Make the dependency explicit and durable rather than relying on timing. The simplest robust approach is manifest-driven orchestration: when pipeline A completes, it writes a manifest row (or updates a status) with pipeline_a_run_id, status=SUCCESS, and ingest_id. Pipeline B's starter checks that manifest for SUCCESS for the required ingest_id before starting. This avoids race conditions and accidental double-runs.

If you use Composer/Airflow, model the dependency in the DAG graph: create a DAG that triggers pipeline A as a task and then, on success, triggers pipeline B (either by calling Data Fusion REST API, publishing to a Pub/Sub topic that B listens to, or triggering a separate DAG). Airflow gives native operators and XComs so you can pass parameters and ensure strict ordering, retries, and SLA enforcement. Use ExternalTaskSensor or TriggerDagRunOperator when DAGs live in separate Composer environments.

Alternatively, use event-driven coupling with Pub/Sub: pipeline A publishes a pipeline-A-complete message (with ingest_id and status) and pipeline B subscribes to that topic or a dedicated starter function does the gating. The starter should validate the success state (manifest + job details) before launching B to avoid false starts from duplicate messages. This is lightweight and scales well for many chained pipelines.

For cross-project or cross-team flows, prefer a centralized orchestrator (Composer) to keep visibility and retries consolidated. Regardless of method, implement idempotency: pipeline B should check for existing completed outputs (manifest, table partition) and skip work if already processed. Also store run metadata and job ids so you can trace the whole chain in logs and dashboards.

Finally, add monitoring and alerts for missed dependencies - alert if pipeline A succeeds but B hasn't started within an expected window, or if B fails repeatedly after A's success. Provide a safe manual replay tool (re-trigger pipeline B for a specific ingest_id) so operators can recover without touching raw inputs.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

**15. Your team uses Airflow. How would you integrate Cloud Composer with Data Fusion pipelines?**

Think of Composer as the orchestration brain and Data Fusion as the ETL worker. The most direct integration is to have Airflow tasks call Data Fusion's REST API (or use gcloud/client libraries) to create/run a pipeline template with conf parameters. In Airflow you'd implement an operator (or use SimpleHttpOperator/custom operator) that triggers the Data Fusion pipeline, captures the returned run_id, and then polls or listens for completion to decide success/failure. Record run_id in XCom so subsequent tasks can reference it.

A decoupled alternative is event-driven: Composer publishes a start message (Pub/Sub) that Data Fusion consumes to begin processing, and Data Fusion publishes completion messages back to Pub/Sub that Composer DAGs listen for (via sensors). This reduces synchronous waits and is useful when you want the DAG to continue with other tasks while Data Fusion runs asynchronously. Use Pub/Sub sensors or a small Cloud Function sensor to bridge the gap.

For reliable error handling and retries, wrap Data Fusion runs in idempotent patterns: the DAG should write a manifest row before triggering the pipeline and verify final output (manifest status, target table partition) after completion. Use task retry policies in Airflow for transient failures, but keep retries bounded and rely on the Data Fusion job-state for the final decision. Also tag Data Fusion runs with labels (dag_id, run_id) to make cross-system tracing easier.

Leverage Composer's UI and scheduling strengths: schedule complex multi-step workflows in Airflow where Data Fusion handles heavy transforms, and use Airflow's SLA, alerting, and dependency features to manage the end-to-end flow. For long-running Data Fusion jobs, use async triggers in Airflow (TriggerDagRun pattern) so the scheduler isn't blocked waiting for external jobs. Store logs and lineage in central places (Cloud Logging, Data Catalog) and connect them via correlation ids.

Finally, standardize operators and templates: create an Airflow operator library that encapsulates Data Fusion crud (launch, poll, cancel, get logs) and document parameter conventions. This reduces boilerplate in DAGs, enforces manifests and idempotency, and makes the integration predictable and easy for other teams to use - the kind of maintainable design interviewers like to hear.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

**16. You need to load data from Salesforce into BigQuery using Data Fusion. Which plugin would you use and how?**

In this case I'd use the Salesforce Source plugin in Data Fusion. This plugin is built for connecting to Salesforce via its APIs and can extract objects like Accounts, Contacts, Opportunities, or even custom objects. In Studio I'd configure the plugin by providing Salesforce credentials - usually a username, password, and security token, or better, an OAuth connection set up through GCP Secret Manager for security. The plugin also lets me choose whether I want to pull the entire object, specific fields, or run SOQL queries for filtered extracts.

Once I've set up the Salesforce source, I'd add transformations if needed - for example, a Wrangler or Projection plugin to rename fields, filter rows, or standardize data types. This is important because Salesforce fields may come in as long strings or nested structures, and BigQuery requires strict typing. At this stage I'd also add metadata like ingest_time or source_system so downstream reporting is easier.

For the target, I'd use the BigQuery Sink plugin, which connects natively to BigQuery. In its configuration I'd provide the target dataset and table, choose write semantics (append, truncate, or upsert using MERGE), and map the field schema accordingly. If I expect large volumes, I'd configure the sink to use batch load jobs from GCS - this is more cost-efficient and scalable than streaming.

The pipeline would typically run on a batch runtime (Spark or Dataproc) because Salesforce data often comes in daily/hourly extracts, and batch makes sense for pulling large amounts of records. If the requirement was near-real-time, I'd consider incremental extracts by using SOQL queries with SystemModstamp or LastModifiedDate filters to pull only new or updated rows.

So, in short, I'd configure Salesforce Source → optional Wrangler/Transform plugins → BigQuery Sink. I'd secure the credentials with Secret Manager, run the pipeline in batch mode, and schedule it in Data Fusion (or Composer) to run at the required frequency. This setup makes the integration simple, auditable, and maintainable.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

**17. You're asked to join two datasets in-flight before writing to BigQuery. Which plugin would you use?**

For joining datasets inside Data Fusion, I'd use the Joiner plugin. The Joiner lets you take two inputs (say, a Salesforce extract and a reference table from GCS or BigQuery) and perform joins much like SQL - inner, outer, left, right joins. You can specify join keys and select which fields to include from each side.

In Studio, I'd connect the two source nodes (for example, Salesforce Source and GCS File Source) into the Joiner. Inside the plugin config, I'd define the join condition (e.g., AccountId in Salesforce = AccountId in reference file), select join type (inner if I want only matches, left if I want all Salesforce rows with possible enrichments), and decide how to handle field name collisions. This step effectively does the enrichment or lookup inline in the pipeline.

After the join, I'd usually pass the data through a Wrangler or Schema Validator to make sure the schema is clean for BigQuery. Then I'd send the output to the BigQuery Sink plugin, which writes to the final dataset. If volumes are big, the pipeline should run on Spark or Dataproc runtime so the join can scale across workers; Direct runtime might work for small reference datasets but isn't suited for production-scale joins.

It's also good practice to partition your join inputs properly. For example, if one dataset is small (a lookup), broadcast joins are efficient. If both are large, make sure the pipeline runs in Spark with enough resources to handle shuffle and avoid OOM.

So the direct answer is: I'd use the Joiner plugin, configure the join type and keys, then output to BigQuery Sink. This gives an in-flight join without having to first land both datasets in BigQuery and then write SQL - a cleaner, ETL-style solution right inside Data Fusion.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

**18. Your pipeline must output data to both BigQuery and Pub/Sub. How would you design this?**

I'd treat the two sinks as independent, single-responsibility consumers of the same validated stream rather than trying to do one atomic multi-sink write. In practice the pipeline first validates and normalizes incoming records, attaches provenance (ingest_id, event_id, correlation_id, ingest_time), and writes the canonical cleaned row to a staging area (staging table in BigQuery or a staging GCS path) or to an internal in-memory stream inside Data Fusion. That canonical record is the source of truth you can replay from.

From that canonical stage I'd fan out to the two sinks. For BigQuery use the BigQuery Sink (batch loads for high-volume or Storage Write API for low-latency), ensuring idempotency via ingest_id + MERGE into the final table. For Pub/Sub use the Pub/Sub Sink (or a small Cloud Run publisher) to publish an event envelope containing the fields downstream systems need plus the provenance metadata. Keep both flows separate so a failure in Pub/Sub publishing doesn't automatically abort a successful BigQuery load.

Handle failure modes explicitly: if BigQuery load succeeds but Pub/Sub publishing fails, write the Pub/Sub intent to a "pending-publish" queue (table or GCS file) and have a retry worker (Cloud Run or Data Fusion lightweight task) drain and republish. Conversely, if Pub/Sub publish succeeds but BigQuery load fails, keep the canonical staging copy and run a retry/reconciliation job. Use a manifest table that records for each ingest_id the status of each sink (BQ_status, PUBSUB_status) so you have a clear audit trail and can reprocess only the failed leg.

Make it observable and idempotent: emit metrics for per-sink success/failure, include ingest_id in every sink message/row, and provide a replay admin endpoint that can re-run failed legs for a given ingest_id. Secure the flow (least-privilege service accounts for BigQuery and Pub/Sub, secrets in Secret Manager) and ensure message schemas are versioned so producers/consumers can evolve safely.

Finally, document the eventual-consistency semantics for stakeholders: explain that writes to both sinks are "at-least-once and eventually consistent" unless you implement a two-phase commit (complex and rarely worth it). Offer an optional compensating workflow (consumer-side dedupe + manifest-driven reconciliation) if the business needs stronger guarantees.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

**19. A team needs a custom connector for an internal API. How would you implement it?**

First pick the right integration surface: for Data Fusion the cleanest long-term approach is to implement a native Data Fusion/CDAP plugin (source or sink) using the CDAP SDK in Java (or the supported plugin SDK). That plugin encapsulates authentication, paging, rate-limiting, error classification, and schema mapping so pipeline authors get a first-class connector in Studio. Build it with configuration options (base URL, auth type, page size, backoff settings, field mappings) and surface schema inference where possible.

If building a plugin is heavy for the team, a pragmatic alternative is to create a small HTTP-based microservice (Cloud Run) that encapsulates the API specifics and exposes a simple REST contract (pull batches, transform payloads). Then use Data Fusion's HTTP/REST plugin (or a generic connector) to call your microservice. This decouples connector logic from Data Fusion and makes testing and deployment easier. Either way, centralize auth (use Secret Manager / Workload Identity) and implement secure TLS and audit logging.

Implement resilience features up front: retries with exponential backoff and jitter, circuit-breaker behavior for sustained failures, configurable concurrency and rate limits to honor the internal API, and a DLQ path for poisoning records. Add batch-mode support (cursor or offset) and incremental-mode support (last-modified token) so consumers can do both full extracts and incremental syncs. Also include pagination and parallel partitioning if the upstream API supports sharding keys.

Build good developer ergonomics: unit tests that mock the API, integration tests that run against a staging instance of the internal API, and a CI/CD pipeline that builds, runs tests, and publishes the plugin artifact to your Data Fusion instance (or deploys the microservice image to Cloud Run). Document connector configs, expected schemas, error codes, and a runbook for on-call. Provide sample pipelines and a schema mapping template to help analysts onboard quickly.

Lastly, plan lifecycle and ownership: assign an owning team for the connector, version the connector artifacts and keep a compatibility matrix, and expose telemetry (latency, error rates, throughput) so consumers can monitor health. This gives you a robust, reusable connector that's maintainable and secure.

**20. A plugin version upgrade breaks your pipeline. How would you manage plugin lifecycle safely?**

Treat plugin upgrades like code releases: version pinning, staged rollout, automated testing, and a quick rollback path. First, pin plugin versions in pipeline artifacts and CI - don't allow autopmatic in-place upgrades in production. When a new plugin version arrives, run a battery of automated smoke and regression tests (unit and integration) against a staging Data Fusion environment using representative sample data. The CI should surface breaking changes early.

Do a canary deployment: upgrade the plugin in a dev/staging instance and run several real-world pipelines (or a sampled subset of production data) to detect regressions in behavior, performance, and schema. Capture performance metrics and compare against baseline (latency, memory, row counts). If the plugin is non-backward-compatible, provide a migration guide and an opt-in feature flag in pipeline configs so teams can migrate at their own pace.

Maintain rollback and artifact management: keep archived copies of previous plugin artifacts and pipeline JSONs so you can quickly re-pin pipelines to the last known-good plugin. Automate rollback in CI/CD (re-deploy the old plugin artifact and re-create pipelines that reference it). Also store pipeline versions in VCS so you can redeploy the exact pipeline state that worked before.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

Add safety nets in runtime: run pipelines under a feature-flag or non-production shadow-run for a release window (execute but don't promote outputs to production datasets) to ensure semantics are preserved. Add post-deploy smoke checks that validate row counts, key metrics, and data checksums - if any assertion fails, auto-disable the new plugin or revert the pipeline and notify owners. Keep a well-documented runbook for upgrade failures.

Finally, communicate and govern: maintain a plugin compatibility matrix, announce planned upgrades with migration notes, and require owners to sign off for breaking semantic changes. Treat plugin lifecycle as part of your release management process - automation, testing, staged rollout and a fast rollback are the practical measures that keep production stable and give teams confidence to upgrade safely.

**21. You receive a CSV file with inconsistent formatting. How would you clean it before loading into BigQuery?**

The first thing I'd do is validate the file as soon as it lands. In Data Fusion I'd use the GCS File Source plugin to pull the CSV into a pipeline, but since the formatting is inconsistent, I wouldn't push it straight to BigQuery. Instead, I'd add a Wrangler transform stage - Wrangler is designed for exactly this kind of cleansing. It lets me visually inspect a sample of rows and apply transformations like trimming spaces, fixing delimiters, normalizing quotes, or handling missing headers.

For example, if some rows use commas and others use semicolons, I can normalize delimiters. If extra columns show up inconsistently, I can drop them or rename them. If certain numeric fields are stored as strings with currency symbols, Wrangler can strip those out and cast to numeric. I'd also standardize date/time formats at this step, because BigQuery requires consistent timestamp or date types.

After the Wrangler step, I'd run a Schema Validation transform to enforce the expected schema. That ensures rows that still don't fit the schema are flagged. Instead of discarding them silently, I'd route invalid rows into a separate sink - usually a GCS rejects bucket or a BigQuery "rejects" table - so I can investigate and fix them later. That way, good rows still make it into BigQuery, and data quality issues are visible.

Once the data is cleaned, I'd use the BigQuery Sink plugin to write the curated data into a staging or final table. For production, I'd configure partitioning and clustering in BigQuery so downstream queries are efficient. To make the pipeline repeatable, I'd parameterize file paths and schema definitions so that every CSV drop can be processed consistently without manual rework.

So overall: GCS Source → Wrangler → Schema Validation → BigQuery Sink, with rejects handled separately. This flow lets me reliably clean inconsistent CSVs and ensures BigQuery only receives high-quality, typed data.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

**22. You need to remove nulls and split fields in input data. Which Data Fusion feature would you use?**

For this scenario, I'd use the Wrangler transform in Data Fusion. Wrangler is specifically built for data preparation and light transformations before loading into a warehouse. It has built-in directives for both tasks - removing nulls and splitting fields.

To remove nulls, I can apply a Wrangler directive like drop row if column_name is null or replace nulls with default values (set column_name default 'UNKNOWN'). This ensures downstream sinks don't choke on missing data or cause inconsistent analytics. If I want to preserve nulls for auditing, I can instead flag them in a separate column.

To split fields, Wrangler supports splitting based on delimiters. For example, if I have a field like "City,State,Zip", I can apply a split-to-columns directive on the comma delimiter and automatically create three new columns. It also supports regex-based splitting if the delimiter is more complex. This is particularly useful for semi-structured text fields.

The nice part is Wrangler lets me do this visually in Studio - I can apply the transformations on a sample and see the results instantly, then generate the script so it runs automatically in the pipeline. After that step, the cleaned schema is enforced, and I can continue to sinks like BigQuery.

So the direct answer: I'd use the Wrangler transform plugin to remove nulls and split fields. It's the easiest and most efficient feature in Data Fusion for this kind of light, record-level transformation.

**23. You need to join two large datasets on multiple keys. How would you handle it in Data Fusion?**

First, I'd look at the data sizes, key cardinality, and whether the join keys are evenly distributed. In an interview I'd say I start by profiling both datasets - checking row counts, key uniqueness, and skew. That tells me whether a straight inner/left join will work or if I'll need to repartition or sample first.

Second, I'd prefer to push heavy joins down to a scalable engine whenever possible. In Data Fusion that means using Wrangler/Transform to prepare keys and then using a Spark engine (or a BigQuery pushdown if the data already lives there) so the join happens where compute is cheapest and data movement is minimal. I'd avoid pulling everything into the pipeline memory.

Third, I'd handle skew explicitly: if a key is very hot I'd salt the join key (add a hash suffix) on the large side and replicate smaller side partitions as needed, or use a broadcast join only when the smaller dataset fits comfortably in executor memory. I'd also increase parallelism and adjust executor memory for the join stage to prevent OOMs.

Fourth, I'd ensure keys are normalized (trim, case, type) and pre-validated so the join condition is deterministic. I'd also consider partitioning the data on the join keys and writing intermediate results to partitioned files if the downstream consumers need them - that helps with re-runs and debugging.

Finally, I'd add monitoring and sampling checks: compare join counts, record loss, and a few key-value spot checks. If the join is recurring, I'd template it with parameters for shuffle partitions and broadcast threshold so we can tune without changing logic.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

**24. Duplicate records are appearing in your output table. How would you fix this?**

First, I'd confirm whether duplicates are real or a symptom of upstream retries, idempotency issues, or late-arriving data. I'd check pipeline logs and source uniqueness keys to see if the duplicates came from repeated ingestion or from the transform logic itself.

Second, I'd implement deduplication with a deterministic key and a clear definition of what makes a record unique (single composite key or a combination of fields). In Data Fusion I'd use an aggregation step (group by unique key) keeping the latest or preferred record based on a timestamp/version field, or use a distinct transform when appropriate.

Third, I'd enforce idempotency at write-time where possible: write to a staging table and run a single upsert (merge) into the target using the unique key, or use BigQuery's MERGE if the target is BigQuery. This prevents duplicate inserts caused by retries and makes reprocessing safe.

Fourth, I'd add schema and data quality checks before writing: assert uniqueness on the key (or fail the job), and emit metrics when duplicates are detected so the team can act. If duplicates are coming from the source, I'd work with producers to add dedupe flags or sequence numbers.

Finally, I'd automate regression tests to catch re-introduced duplication and document the source-of-truth key and dedupe policy so future engineers know how duplicates are handled.

**25. Your team debates whether to transform data in Data Fusion or push down to BigQuery. How would you decide?**

I'd base the decision on data locality, scale, cost, and complexity. If the data already lives in BigQuery and transformations are SQL-friendly (aggregations, joins, window functions), pushing down to BigQuery is usually simpler, faster, and cheaper because you avoid data movement and leverage BigQuery's scalable execution.

If transformations are complex, require custom logic, or need integration with non-BigQuery sources (APIs, files in GCS with heavy parsing), Data Fusion (Spark) can be a better choice because it's more flexible and supports complex ETL patterns. Also if you need reusable pipelines, connectors, or orchestrated steps, Data Fusion adds operational value.

I'd also consider cost model and performance SLAs: BigQuery charges for processed bytes and can be extremely fast for analytical SQL, while Data Fusion (Spark) has cluster costs and longer development cycles for complex transforms. For small-to-medium transformations where SQL suffices, prefer BigQuery; for heavy joins across many external sources or CPU-bound processing, prefer Data Fusion.

Another factor is maintainability and skills: if the team is strong in SQL and less in Spark, keeping logic in BigQuery simplifies ownership and troubleshooting. Conversely, if you need versioned pipelines, complex stateful transforms, or custom libraries, Data Fusion wins.

Finally, I'd prototype both approaches on a representative dataset, compare runtime, cost, and correctness, then choose based on those concrete results and document the reasons so the team has a repeatable decision rubric.

© Shubham Wadekar

**All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar**

**26. You must load data into a partitioned BigQuery table daily. How would you configure Data Fusion?**

I'd start by choosing the partitioning strategy - typically a date-based partition on an ingestion timestamp or a business date column - and make that decision explicit in the design. In the pipeline I'd make sure the partition column is present and normalized (type DATE or TIMESTAMP) and, if needed, add a derived column like event_date using a transform so every row can be mapped to the correct partition.

Next, I'd configure the BigQuery sink/plugin in Data Fusion to write into the partitioned table. For daily batch loads I prefer writing batch files to GCS (partitioned by date in path if useful) and then using a BigQuery load job or the BigQuery sink's load mode that targets the table's partition decorator (e.g., mydataset.mytable$YYYYMMDD) or sets the timePartitioning.field. This keeps files small, predictable, and avoids hot single-row streaming costs.

Operationally I'd schedule the pipeline to run once per day (or with a small SLA window) and make the partition date an explicit runtime parameter so the same template works for backfills. I'd also set write-disposition semantics (append vs overwrite per-partition) - usually append with a controlled swap: write to a staging partition and then atomically swap or run a MERGE if dedupe/upsert is required.

I'd enable partition expiration and clustering where appropriate (for query performance), and add pre-flight checks so the pipeline fails fast if the incoming partition has no data or schema changes. Finally, I'd add monitoring and metrics: row counts per partition, load job success/failure, and a small sample validation comparing source counts to inserted rows so we quickly detect missing partitions or partial writes.

**27. The team wants real-time data in BigQuery. Would you use streaming or batch ingestion? Why?**

If the requirement truly needs low-latency reads in seconds (real-time dashboards, fraud alerts), I'd choose streaming ingestion - typically Pub/Sub → Dataflow (or Data Fusion streaming) → BigQuery streaming inserts - because it delivers data quickly and supports sub-second to second latency use cases. Streaming fits when users need near-immediate visibility and can tolerate eventual consistency quirks of streaming inserts.

If the product accepts a small delay (minutes) or the volume/cost profile prefers it, I'd pick micro-batch: collect events in Pub/Sub or GCS for a short window (1–15 minutes) and run a batch load into partitioned BigQuery tables. Micro-batches reduce streaming costs, improve load efficiency, and simplify strong consistency and dedup semantics (load jobs + MERGE are easier to make idempotent).

I'd weigh tradeoffs: streaming costs more per row and has quotas/ingestion limits and slightly different semantics (e.g., possible duplicates, partition buffering), while batch gives cheaper, consistent loads and easier schema evolution. Also consider downstream query patterns: if analysts run many large scans, batches are cheaper because you can compact data before it's queryable.

Operationally I'd prefer streaming only if the business needs justify it; otherwise micro-batch often hits the sweet spot between latency, cost, and reliability. Whatever the choice, I'd design for idempotency (dedupe keys, sequence numbers), monitoring for lag/error, and a fallback reprocessing path so we can recover from failures without data loss.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

**28. Your pipeline schema doesn't match the target BigQuery table. How would you handle mapping?**

First, I'd profile and document the difference: which fields are extra/missing, type mismatches, nullable vs required, and any semantic changes. In the pipeline I'd add an explicit mapping/transformation stage that renames fields, casts types, drops unused fields, and derives missing columns (with sensible default values), so the output conforms exactly to the BigQuery schema.

For type differences I'd perform safe casts and add validation checks - e.g., parse timestamps with fallback rules, validate numerics and emit bad-row logs to a dead-letter bucket rather than failing the whole job. If BigQuery requires strict schema (non-nullable), I'd either backfill defaults or change the target to nullable while coordinating with consumers to avoid breaking queries.

If schema change is frequent, I'd use a controlled schema evolution process: stage incoming data into a raw table (schema-on-write or permissive JSON column), run a scheduled schema-mapping job to transform and validate, and then MERGE into the production table. That gives us time to adapt to producer changes and keep production schema stable.

Finally, I'd add automated contract tests (CI) and monitoring: schema drift alerts, row-level errors, and sample validations. I'd document the mapping rules and own backward/forward compatibility decisions with stakeholders so future producers/consumers know how schema mismatches are handled.

**29. You want aggregations performed inside BigQuery rather than Data Fusion. How would you design this?**

I'd start by keeping the pipeline in Data Fusion lightweight: extract, clean, normalize keys and types, and write the raw or minimally-transformed data into a staging table in BigQuery. This keeps the heavy compute where it belongs and avoids moving large volumes of data back and forth; Data Fusion's job is framing and delivery, not doing the analytic work.

Next, I'd design a set of parameterized SQL aggregation jobs in BigQuery - either scheduled (scheduled queries) or triggered (Cloud Scheduler + Cloud Functions calling the BigQuery API) depending on latency needs. Each job would read from the staging table, perform the aggregations with well-tested SQL (GROUP BY, window functions, approximate aggregation if needed), and write results to the analytics/summary tables (partitioned & clustered for performance).

I'd make the SQL idempotent and safe for re-runs: use MERGE for upserts into the final tables, or write to a dated partition and then swap/merge so retries don't create duplicates. Also include audit columns (ingestion_time, source_batch_id) so you can trace which staging rows contributed to an aggregate run and support reprocessing if necessary.

Operational controls are important: expose parameters (date range, partition) as runtime inputs to the scheduled queries or orchestrator, track job metrics (bytes processed, runtime), and set query cost limits. I'd also add lightweight materialized views or partitions for frequently-accessed aggregates to reduce repeated compute costs for dashboards.

Finally, I'd version and test the SQL in source control with sample datasets and CI checks. Document aggregation logic clearly (business definitions, keys, null-handling) so analysts and engineers trust the output and can update aggregates when business rules change.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

**30. New columns appear in source data. How would you handle schema evolution when writing into BigQuery?**

First, I'd accept the reality that producers will change and treat incoming data as semi-structured at the ingest boundary. I'd land the raw inputs into a permissive staging area in BigQuery - either a table with a wide nullable schema or a single JSON/STRING column storing the raw row - so new fields don't immediately break the pipeline.

Second, I'd implement a controlled schema-mapping layer that runs after ingest: a transformation SQL or Data Fusion job that reads staging data, extracts known fields, safely casts types, and either drops or archives unknown columns into a separate "extensions" column (JSON) or a metadata table. This lets us preserve new data for future use without destabilizing production schemas.

Third, formalize an evolution process: automated discovery that detects new columns (alerts with example values), a lightweight review workflow (do we map, ignore, or promote the field?), and a migration plan if the field becomes part of the canonical schema. Use CI tests that fail builds when a breaking change is introduced unexpectedly, and require an explicit schema-change PR for production changes.

Fourth, when promoting new columns into the production table, prefer nullable columns or add them as optional with defaults, and run a backfill job where needed. If write patterns require strict schemas (non-nullable), coordinate with consumers and use feature flags or staged rollouts so queries don't fail in downstream dashboards.

Finally, add observability: schema-drift metrics, row-level error logs (dead-letter), and sampling so stakeholders can see how often the new columns appear and whether they're meaningful. Document mapping rules and keep the evolution policy visible so producers know how to introduce fields safely.

**31. You must process Pub/Sub events and store results in BigQuery. How would you build this?**

I'd design an event-driven pipeline using Pub/Sub → Dataflow (or Data Fusion streaming) → BigQuery. Pub/Sub is the ingestion backbone; Dataflow gives me windowing, exactly-once semantics with proper buffering, and built-in connectors to BigQuery so I can handle transformations, enrichment, and deduplication before writes. I'd pick Dataflow when I need low latency and strong streaming semantics, and Data Fusion streaming if the team prefers the UI and ready connectors.

In the streaming job I'd parse and validate each message up front - schema validation, required fields, type casts - and send malformed messages to a dead-letter Pub/Sub or GCS bucket with context for debugging. For correctness I'd add an event-id or sequence number if producers don't already provide it, so I can dedupe before writes using an in-memory windowed dedupe or by using BigQuery's insertId/merge patterns.

For BigQuery I'd avoid tiny streaming inserts where possible; either use streaming inserts with insertId to prevent duplicates for true near-real-time needs, or buffer into short micro-batches and run load jobs into partitioned tables (this reduces cost and improves consistency). I'd design the BigQuery schema as partitioned & clustered and include audit columns (ingestion_time, source_offset, batch_id) so downstream can trace back events.

Operationally I'd add monitoring and alerting for subscription lag, Dataflow job health, dead-letter volume, and BigQuery write errors. I'd also write idempotent upsert logic (MERGE) or use partitioned staging tables plus scheduled dedupe/merge jobs so reprocessing is safe. Finally, I'd keep the transformation logic versioned in source control and expose runtime parameters (window sizes, offsets, table names) so we can roll forward/back without rebuilding the job.

© Shubham Wadekar

**32. You need to publish transformed results from Data Fusion back into a Pub/Sub topic. How would you do it?**

I'd add a Pub/Sub sink step at the end of the Data Fusion pipeline and ensure the output messages are compact, schema-validated, and include metadata (source id, transform version, timestamp). If Data Fusion's native Pub/Sub plugin is available and reliable for our throughput, I'd use that; otherwise I'd write transformed records to GCS or BigQuery and have a lightweight publisher (Dataflow or Cloud Function) read those and publish to Pub/Sub - this decouples transformation from publish for better reliability.

Before publishing I'd serialize messages consistently (JSON or Avro) and include a schema version field so downstream consumers can evolve. I'd also implement retry/backoff and exponential retry policies at the publisher to handle transient Pub/Sub errors, and route permanently failing messages to a dead-letter topic with context so we don't lose data silently.

If ordering matters, I'd leverage Pub/Sub ordering keys or design the pipeline to keep ordering within a partitioned key. For at-least-once semantics I'd include an idempotency token (message id) so consumers can safely dedupe. I'd also make publishing asynchronous if throughput is high: buffer small batches and publish them to reduce RPC overhead while respecting latency requirements.

Finally, I'd add monitoring for publish success rates, topic quotas, and consumer lags, and document the message contract (schema, semantics, versioning) so teams can consume the transformed stream without surprises.

**33. Pub/Sub messages change schema over time. How would you adapt the pipeline?**

I'd treat schema evolution as expected and design the pipeline to be tolerant: first, enforce a lightweight schema validation step that detects new/unknown fields and routes unexpected variants to a staging area for inspection rather than failing the whole pipeline. That preserves data while we decide how to handle the change.

Second, I'd include a schema version in every message and keep a centralized schema registry or simple metadata table that maps versions to parsing logic. The transform layer reads the version and applies the correct parsing/mapping. For minor additive changes I'd parse into a flexible structure (e.g., a JSON column or an extensions map) so we can preserve new attributes without immediate production schema changes.

Third, I'd automate detection and alerting: whenever new fields or incompatible type changes appear, send a notification with sample messages and suggested mapping. That triggers a lightweight review workflow where product and data teams decide to map, ignore, or promote the field into the canonical schema.

Fourth, when promoting fields, I'd make changes backward-compatible - add nullable columns or extension fields first, run a backfill if needed, and version the transform logic. I'd also add CI tests that validate how each schema version will be parsed so deployments don't accidentally break older producers or consumers.

Finally, I'd document the evolution policy and the message contract, keep historical raw messages in a landing area for reprocessing, and ensure consumers are resilient (they should ignore unknown fields). This combination of detection, flexible ingestion, explicit versioning, and governance keeps the streaming system robust as schemas change.

© Shubham Wadekar

**All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar**

**34. Some messages fail to process. How would you use dead-letter topics to handle them?**

I'd treat dead-letter topics (DLQs) as part of normal flow, not an error graveyard. Upfront I'd classify failures: transient (temporary downstream errors), transient-but-retriable (schema timeouts, API flakiness), and permanent/poison (bad format, corrupt payload). For transient errors I'd rely on retries with exponential backoff and only send to the DLQ after the retry budget is exhausted. For permanent errors I'd move the message to the DLQ immediately with an error reason so it's clear why it failed.

When publishing to the DLQ I'd include rich context - the original message, processing metadata (timestamp, subscription id, pipeline version), and an error code + human-readable reason - so investigations and automatic reprocessing can use that metadata. I'd store the same payload also in a raw archive (GCS) for long-term auditing and to support complex reprocessing that a simple DLQ topic might not be ideal for.

Operationally I'd set retention and access controls: short retention on the DLQ (to avoid noise) but longer retention for archived raw messages; restrict who can publish/subscribe to the DLQ; and expose a small dashboard showing DLQ rate, top error reasons, and sample messages. Alerts should fire when DLQ rates spike or when the DLQ holds messages older than an agreed SLA so the team treats it as a production incident rather than background noise.

For reprocessing I'd build reproducible playbooks: automated replay tools that read DLQ messages, apply fixes or schema-upgrade logic, and republish to the original topic or to a special reprocess topic. Each replay should be idempotent (use message ids) and track provenance so we don't double-insert into BigQuery. For complex fixes I'd prefer manual review + scripted replay to avoid amplifying bad data.

Finally, I'd bake DLQ behavior into CI/CD and runbooks: tests that simulate failures and ensure messages end up in DLQ with correct metadata, runbooks for engineers to triage and replay, and periodic audits to close the loop with producers if the same poison messages keep appearing. This keeps the pipeline resilient, observable, and fixable.

**35. You're asked to design a real-time Pub/Sub → Data Fusion → BigQuery pipeline. What would be your approach?**

I'd start by clarifying latency and correctness SLAs: do we need sub-second visibility or is a few-seconds/minutes acceptable? That drives choices like pure streaming inserts versus micro-batching. With strict real-time needs I'd design a streaming Data Fusion (or Dataflow if low-latency semantics are required) job that subscribes to Pub/Sub, parses and validates messages, enriches or joins as needed, then writes to BigQuery using streaming inserts with insertId for dedupe. If slightly relaxed latency is fine, I'd buffer short micro-batches to use load jobs for cost-efficiency and stronger consistency.

In the transform layer I'd focus on defensive coding: schema validation, type casting with fallbacks, and routing malformed messages to a dead-letter topic or staging GCS. I'd add deduplication using event IDs or sequence numbers (windowed dedupe in streaming) and handle hot keys (salting or parallelization). I'd also include a small enrichment/cache layer (in-memory or external lookup) only if necessary to avoid external call latency in the critical path.

For BigQuery writes I'd prefer partitioned & clustered tables and choose write strategy based on latency vs cost: streaming inserts for immediate visibility, or short-window load jobs + MERGE for better cost and idempotency. Whichever we choose, writes must be idempotent - include an audit batch_id or use insertId with MERGE semantics - so retries or replays don't create duplicates. I'd also add schema-versioning metadata so downstream users know which transform version produced each row.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

Operational concerns get equal weight: autoscaling for the streaming job, monitoring for Pub/Sub subscription lag, processing latencies, DLQ growth, and BigQuery insert errors. I'd set up alerts for lag thresholds, error rate spikes, and SLO breaches. Also plan capacity: Pub/Sub quotas, BigQuery streaming quota implications, and Data Fusion cluster sizing or Dataflow worker autoscaling rules.

Finally, I'd validate the design with a proof-of-concept on representative traffic, compare costs and latency for streaming vs micro-batch, and iterate. I'd keep everything in version control, add CI tests that simulate failover and schema drift, and document runbooks for replay and emergency backpressure strategies so the pipeline is both performant and operable.

**36. You get daily JSON files in GCS. How would you ingest them into BigQuery using Data Fusion?**

I'd build a simple, reliable Data Fusion pipeline that treats the GCS location as the canonical landing zone. Use the GCS source plugin to read the JSON files (newline-delimited JSON if possible), and add an early parsing/validation transform to catch malformed rows and route them to a dead-letter GCS path. Keeping raw input intact is important - I'd also copy incoming files to a raw-archive folder with a timestamped path for provenance.

Next, normalize and clean the records: apply a transform stage that enforces types, parses nested JSON into flat fields if needed, and derives any partition column (like event_date) from the payload or filename. I'd keep an "extensions" JSON field for unknown or new attributes so we don't lose data when schema drifts happen. Add lightweight row-level validation (required keys, timestamp sanity) and increment counters for monitoring.

For the BigQuery write, I prefer writing to a staging table first (partitioned and clustered appropriately). Use the BigQuery sink in Data Fusion to load the cleaned data into that staging table, and then run a controlled MERGE from staging into the production table to handle dedupe/upserts and enforce schema constraints. This two-step (stage → MERGE) pattern makes replays safe and keeps production tables consistent.

Operationally, parameterize the pipeline (GCS path, partition date, staging table) so it's reusable for backfills, and schedule it daily via Cloud Composer or Cloud Scheduler. Add monitoring for file counts, row counts, and load job status; alert on mismatches between expected and actual rows. Keep good logging for any rows that hit the dead-letter path so engineers can triage quickly.

Finally, document the contract: file naming, expected JSON schema, partitioning rules, and recovery steps. I'd also add a small post-load validation - compare source file row counts (or recorded row_count metadata) to rows landed in BigQuery for the partition - so we detect partial loads early and can rerun safely.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

**37. Only new files should be processed - how would you implement incremental file ingestion?**

I'd avoid full directory scans on every run and instead rely on deterministic file discovery. The simplest is to use GCS notifications (Pub/Sub) that trigger the Data Fusion pipeline (via Cloud Function or orchestrator) whenever a new file is created. That way each pipeline run receives the exact object name and only processes that file - zero extra scanning and near-real-time behavior if needed.

If Pub/Sub notifications aren't available, I'd maintain a lightweight file-manifest (a tracking table in BigQuery or Firestore) that records processed object names, generation numbers, and timestamps. The pipeline's pre-step lists objects in the target prefix and filters out already-processed names using the manifest; it only proceeds with new entries and updates the manifest atomically after successful processing.

Another robust pattern is to use atomic marker files or a landing-in-progress folder: producers write to a temp path and then move the file (or write a ".ready" file) to the inbound folder when complete. The pipeline only processes files that have an accompanying ready marker and then moves processed files to an archive folder (or tags them) so they're not picked up again. This avoids partial-file processing and double-processing.

For idempotency, always record a processing outcome (success/failure) and store file checksums or GCS generation numbers in the manifest. On retry or replay, compare checksum/generation to decide whether reprocessing is necessary. Also make writes to BigQuery idempotent (MERGE by a unique key or write to a partitioned staging table) so accidental re-runs don't create duplicates.

Finally, add monitoring and alerts for unprocessed files older than an SLA, manifest inconsistencies, and DLQ growth. Document the discovery mechanism and recovery steps so the team knows how to replay a single file or backfill a date-range without guessing which files have already been processed.

**38. You need to load data into a partitioned BigQuery table from GCS. How would you design the pipeline?**

I'd treat GCS as the canonical landing zone and start with a small, deterministic ingestion step that reads only the new files (GCS notifications or a manifest) and copies them to a raw-archive path for provenance. Keeping the raw copy untouched gives us an audit trail and an easy reprocess source if anything goes wrong.

Next I'd have a Data Fusion (or Dataflow/Spark) transform stage that parses the files, normalizes types, derives the partition column (business_date or ingestion_date) and validates rows. Any malformed rows get routed to a dead-letter GCS location with context so the main job doesn't fail on a few bad records.

For BigQuery writes I prefer a staging → finalize pattern: write cleaned data into a partitioned staging table or to dated files in GCS and then run a BigQuery load job or a MERGE into the production partition. Use partition decorators (e.g., table$YYYYMMDD) or set timePartitioning.field so each run targets only its partition and you can safely re-run without touching other partitions.

Make the process idempotent: record file metadata (name, checksum, GCS generation) in a processing manifest table and only mark files as processed after a successful load + validation. Use MERGE or upsert semantics for dedupe if necessary, and include audit columns (source_file, ingestion_ts, batch_id) in the target so tracing and reprocessing are straightforward.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

Finally, add operational controls: schedule the pipeline (daily or parameterized), expose runtime parameters (partition date, paths), monitor row counts vs source counts, alert on partition-load failures or missing expected files, and keep clear runbook steps for backfills. Partition expiration and clustering on high-cardinality query keys are nice-to-haves for query performance and cost control.

**39. A file schema doesn't match the target schema. How would you handle this?**

First, I'd fail fast but gracefully: detect schema mismatches early in the pipeline during a validation step and route offending rows or files to a staging/dead-letter area rather than letting them corrupt production tables. Capture sample payloads and metadata so the team can quickly understand the difference.

Second, apply explicit mapping/transformation logic: rename fields, cast types, drop unsupported fields, and derive missing columns with defaults where appropriate. Do this in a controlled transform job (Data Fusion/SQL) so the output conforms exactly to the target schema before any write operations are attempted.

Third, for additive or unknown fields I'd preserve them in an extensions column (JSON) or archive them in a raw table so we don't lose potentially useful data. If the change is a breaking structural change, follow a formal schema-evolution process: notify stakeholders, propose a migration (add nullable columns first), and schedule a production schema update with a backfill if necessary.

Fourth, make the process safe for retries and reprocessing: use a manifest and file checksums to avoid double-processing, and do final writes via MERGE/upsert to prevent duplicates. If many producers change frequently, consider landing raw data into a permissive schema-on-write table and transform into the canonical schema in a separate, audited step.

Finally, automate detection and governance: alerts for new fields or incompatible types, CI checks for schema changes, and documentation of mapping rules so producers and consumers are aligned. That way schema mismatches become a managed, observable event rather than a production outage.

**40. You need to write processed data back into GCS in Parquet format. How would you do it?**

I'd use the transform engine (Data Fusion's Spark runner or a Dataflow job) to write Parquet because these engines handle schema, partitioning, and compression efficiently. After transformations and validation, configure the sink to write Parquet files to a designated GCS path, using a consistent directory layout (for example, gs://bucket/data/year=YYYY/month=MM/day=DD/) so downstream jobs and tools can discover partitions easily.

Choose a sensible partitioning and file-size strategy: partition by the main queryable date or key, and aim for parquet file sizes in the 100MB–1GB range to balance parallelism and query efficiency. Use a compression codec like Snappy (default for Parquet) and preserve schema metadata so Parquet readers detect types correctly.

If schema evolution is possible, write an extensions JSON or keep a schema registry/version in metadata files alongside the parquet objects. Also, write success/manifest marker files (e.g., _SUCCESS and a manifest listing file names and checksums) so consumers know when a directory is ready to be read and to support atomic processing semantics.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

For downstream compatibility, validate Parquet output with a small sampling job that reads a few files and compares row counts and key aggregates against the source. Automate this check in the pipeline and fail the run if mismatches exceed thresholds. Finally, document the Parquet contract (path layout, partition keys, column types, compression) and add monitoring for write failures, file counts, and stale partitions so the team can operate and consume the data reliably.

**41. Your pipeline requires heavy transformations on 5TB of data. How would you leverage Dataproc with Data Fusion?**

I'd treat Dataproc as the heavy-lifting execution engine and Data Fusion as the orchestration/authoring layer. Design the pipeline in Data Fusion but configure it to use a Dataproc profile so the expensive transform stages run on an autoscaled Dataproc Spark cluster - this gives me Spark's distributed processing power while keeping the pipeline easy to manage in Data Fusion's UI.

Next, optimize data layout and I/O: store intermediate inputs as columnar Parquet/ORC on GCS, partition by a high-cardinality date/key used in queries, and read/write using the GCS connector to minimize shuffle and network overhead. Use broadcast joins where one side is small, and repartition or salt keys for skewed joins. Tune Spark settings (shuffle partitions, executor memory/cores) via Dataproc cluster initialization or runtime profile to match the 5TB footprint.

For cost and reliability I'd use ephemeral Dataproc clusters with autoscaling and preemptible workers for non-critical work to reduce cost. Keep the master and critical workers non-preemptible, and use cluster templates so jobs spin up with consistent libraries (JARs, python wheels) and config (spark.memory, spark.serializer). Use initialization actions to install any custom connectors or jars Data Fusion will need on the Dataproc nodes.

Operationally, split the job into stages: heavy transforms on Dataproc, write partitioned outputs to GCS, then run lightweight merges/loads into BigQuery if needed. Add checkpoints and write intermediate results to GCS so retries don't re-run everything. Instrument the job with metrics (bytes shuffled, executor GC time, task failures), log sampling, and a small validation step comparing source and output counts to detect silent corruption early.

Finally, validate with a scaled POC - run on a representative subset (say 5–10%) and extrapolate cluster sizing and cost before full run. Keep tuning knobs as runtime parameters so you can increase parallelism or memory without code changes, and document cluster templates, tuning rationale, and runbooks for engineers to reproduce or troubleshoot the Dataproc-backed pipeline.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

**42. The pipeline is small-scale. Would you choose Direct runtime or Spark runtime? Why?**

For small-scale pipelines I'd choose the Direct runtime. It's lightweight, starts fast, costs less to operate, and is simpler to debug - perfect for small volumes and straightforward transforms (field-level cleans, simple joins, basic parsing) where the overhead of spinning up a distributed Spark cluster isn't justified.

Direct runtime is also great for developer productivity: quick iterative edits in Data Fusion, faster template runs, and less operational burden (no cluster sizing, no autoscaling configs). If the transforms are CPU-light and fit in a single machine's memory, Direct reduces latency from orchestration and is cheaper over time.

That said, I'd pick Spark runtime if I expect growth, heavier joins, wide shuffles, or if I need Spark-specific libraries or advanced optimizations. Spark gives better parallelism, fault-tolerance for large-scale sorts/aggregations, and fine-grained tuning - but it adds cost and operational complexity that small pipelines rarely need.

So my decision rule is simple: use Direct for small, predictable workloads with modest SLAs and Spark for data volume, complexity, or performance requirements that exceed a single node. Finally, keep the pipeline modular so you can migrate from Direct to Spark later - author transforms in a way that's portable and parameterized, so scaling up becomes an operational change, not a rewrite.

**43. A transformation step is too compute-intensive. How would you offload it to Dataproc?**

First, I'd isolate and profile the heavy transformation to understand why it's expensive - large shuffles, wide aggregations, heavy UDFs, or skewed keys - so I know exactly what to move and how to optimize it on Dataproc. This profiling tells me whether I need a pure Spark rewrite, a repartition/salting strategy, or just different resource sizing.

Second, I'd refactor that step into a separate job in Data Fusion and configure a Dataproc profile for it. In practice I keep the pre- and post-processing in Data Fusion and run the heavy transformation as a Dataproc-executed Spark transform (or a submitted Spark job) so the work runs distributed across a properly sized cluster while the rest of the pipeline stays lightweight.

Third, I'd tune the Dataproc cluster for that workload: set appropriate worker counts, memory/CPU per executor, and shuffle partitions; enable autoscaling and use preemptible workers for cost savings where acceptable. I'd also package any custom libraries or jars via initialization actions so the job runs consistently and avoid repeated setup time.

Fourth, I'd handle data locality and I/O by using columnar formats on GCS (Parquet/ORC) and partitioning to minimize unnecessary reads, and by staging intermediate outputs to GCS so retries don't re-run everything. For joins, I'd prefer broadcast joins when one side is small, or salting and repartitioning when skew exists.

Finally, I'd add monitoring and validation: track executor metrics, shuffle bytes, GC time, and task failures; add a post-run validation comparing key counts and checksums to the original. After a tuned Dataproc run, I'd iterate on cluster sizing and job parallelism until the cost/performance balance meets the team's SLA.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

**44. Your Dataproc-backed pipeline is too expensive. How would you tune cost/performance?**

First, I'd identify the cost drivers - long-running clusters, oversized executors, excessive shuffle, small file overhead, or inefficient algorithms - by looking at job timelines, cluster uptime, and Spark metrics so I know where to optimize rather than guessing.

Second, I'd move to ephemeral clusters or use Dataproc autoscaling so clusters exist only for job duration. For non-critical workloads I'd enable preemptible workers to save cost, keeping enough non-preemptible capacity for stability. Also compress intermediate data and prefer columnar formats to reduce IO and storage costs.

Third, I'd optimize Spark settings: right-size executors (memory/cores) to reduce overhead, tune spark.sql.shuffle.partitions based on data size, and avoid unnecessary shuffles by reordering transforms or using map-side aggregations. Consolidate small files into larger output files to reduce task overhead and BigQuery load inefficiency.

Fourth, review algorithmic changes: replace expensive UDFs with native Spark SQL, use built-in aggregations, and prefer approximate algorithms (HyperLogLog, approximate quantiles) where business requirements allow. Cache only when necessary and clear caches promptly to free memory.

Finally, implement cost-aware scheduling and monitoring: add query/job cost estimates to CI, track cost per run, set budgets and alerts, and run POCs on sample datasets before full runs. Document the tradeoffs so stakeholders understand performance vs cost, and create templates that encode the tuned settings for repeatable, cheaper runs.

**45. You must migrate Hive jobs from on-prem to GCP. How would you use Data Fusion with Dataproc?**

I'd start by inventorying the Hive jobs: capture queries, UDFs, dependencies, input/output locations, and SLAs. That inventory guides migration strategy - which jobs map directly to Spark SQL, which need UDF porting, and which require refactoring due to external integrations.

Next, I'd re-platform the heavy Hive workloads to Dataproc because it natively runs Spark and can execute Hive-compatible queries. I'd use Data Fusion as the orchestration and templating layer: author pipelines that invoke Dataproc jobs for the heavy Hive conversions while keeping data ingestion and downstream integrations in Data Fusion for operational consistency.

For SQL and UDFs, I'd test converting HiveQL to Spark SQL and port or rewrite UDFs in PySpark/Scala to run on Dataproc. Where possible, lean on Tez-to-Spark or Hive-on-Spark compatibility modes, and validate outputs against known benchmarks. Store intermediate and final datasets in GCS as Parquet/ORC and use partitioning to preserve performance characteristics.

Operationally, I'd use Dataproc init scripts to provision required jars and libraries, version the cluster templates, and use Data Fusion to schedule, parameterize, and monitor jobs. Implement a phased cutover - run in shadow mode (parallel runs) to validate parity, reconcile results, then promote the GCP pipelines to production once stable.

Finally, automate tests and runbooks: create CI that verifies transformed job outputs against legacy Hive results on sample datasets, add monitoring and alerts, and document rollback steps. Communicate with stakeholders about compatibility limits and performance trade-offs so the migration is predictable and auditable.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

**46. You need to replicate data from Cloud SQL into BigQuery in near real time. How would you design this?**

I'd use a CDC-first approach: capture changes from Cloud SQL and apply them to BigQuery rather than doing full-table polls. The practical pattern is to enable a CDC streamer (Datastream is Google's managed CDC service) that connects to Cloud SQL, captures the initial snapshot plus ongoing change events, and publishes those changes to Pub/Sub or a staging GCS location. This gives a reliable, low-latency change feed without putting heavy load on the source DB.

Next, I'd process the change events with a streaming transform layer (Dataflow or a Data Fusion streaming job). That layer parses the CDC format, normalizes column types, enriches or filters if needed, and turns change events into idempotent operations for BigQuery (insert/update/delete). I prefer generating a compact envelope that includes operation type, primary key, commit timestamp, and a source LSN/offset so downstream can dedupe and order safely.

For applying to BigQuery I'd avoid blind streaming inserts when correctness matters; instead write to a partitioned staging table or use streaming inserts with insertId + a downstream MERGE job. A common pattern: stream into a staging table keyed by primary key + commit_ts, then run periodic or near-real-time MERGE statements into the production table (MERGE by primary key using the latest commit_ts). That gives idempotency and supports deletes/updates cleanly while allowing replays without duplication.

Operationally I'd build schema-evolution handling and an initial snapshot/backfill plan: Datastream (or the capture service) must be able to take a snapshot of existing data for the first load and then stream changes. Add monitoring (replication lag, Pub/Sub backlog, Dataflow health), DLQs for malformed records, and alerting for schema drift. Also secure the pipeline with VPC peering or private connectivity, grant least-privilege IAM to Datastream/Dataflow, and encrypt data at rest/in transit.

Finally, document runbooks and test replays: keep raw change events in GCS for auditing and reprocessing, version your transform logic, and provide a rollback/reconciliation plan (row counts, checksums, and a point-in-time comparison) so you can verify parity with Cloud SQL and recover from errors without data loss.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

**47. A client wants to capture CDC from Spanner and load into BigQuery. How would you implement this?**

I'd start by picking the CDC mechanism: for Cloud Spanner there are two common methods - use Datastream as a managed CDC connector (if supported for Spanner in your environment) or use Spanner Change Streams (native Spanner feature) to read change records. I'd prefer Spanner Change Streams when available because they're native, low-latency, and efficient; Datastream can be used when you want a uniform CDC product across multiple sources or need its managed features.

Once changes are emitted (Change Streams or Datastream), I'd push them into a streaming processing layer (Dataflow is typical). The Dataflow job reads the change records, applies schema normalization and business transformations, and converts Spanner change events into idempotent BigQuery operations - again carrying primary key, commit timestamp, and a change-type envelope so merges are deterministic. If using Change Streams, you can read with the Spanner client library and publish events to Pub/Sub for decoupled processing.

For BigQuery ingestion, I'd stream events into a staging table (or directly use streaming inserts with insertId), and use MERGE logic keyed on the Spanner primary key and commit timestamp to apply updates/deletes. This pattern supports exactly-once semantics in practice: the staging table captures all events, and controlled MERGE ensures the production table reflects Spanner's state in order. For high-throughput scenarios, micro-batching the staging-to-merge step can reduce cost while keeping acceptable latency.

Operational items are important: handle schema evolution (keep an extensions JSON or schema registry and a review process), monitor replication lag, watch change-stream positions (so you can resume after failures), and keep raw change payloads in GCS for auditing and replays. Secure access (IAM, VPC) and test end-to-end on representative traffic; also validate data parity periodically (row counts, checksums) to catch missed or out-of-order events.

Finally, choose the implementation based on requirements: if the client needs integrated, low-latency, and simpler operational model, favor Spanner Change Streams + Dataflow → BigQuery. If they already use Datastream for other sources or want managed initial snapshots and uniform tooling, Datastream → Pub/Sub → Dataflow → BigQuery is a solid alternative. In either case, build idempotent, observable transforms, a replayable raw store, and clear runbooks for recovery and schema changes.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

**48. You must ingest IoT time-series data from Bigtable into BigQuery. How would you approach it?**

I'd treat Bigtable as the low-latency, high-ingest store for raw IoT telemetry and BigQuery as the analytics / reporting store. The pipeline's job is to extract relevant time-series windows from Bigtable, transform/aggregate into analytics-friendly rows, and load into partitioned/clustered BigQuery tables optimized for the kinds of queries dashboards and ML will run. That separation keeps operational load where it belongs and avoids making Bigtable serve heavy analytical scans.

For the extractor I'd use Dataflow (or a Data Fusion pipeline that uses a Dataproc/Dataflow-backed transform) with the Cloud Bigtable connector. Dataflow can read ranges or timestamp-bounded scans in parallel, handle backpressure, and perform windowed aggregations or downsampling (e.g., one-minute/one-hour rollups) before writing. If the IoT producer also publishes events to Pub/Sub, I'd prefer real-time streaming from Pub/Sub into BigQuery (via Dataflow) and reserve the Bigtable→BigQuery job for backfills and bulk reprocessing.

Schema and partitioning matter: design BigQuery tables partitioned by event_date (or ingestion_date) and clustered by device_id/region to speed common filters. Convert wide, sparse Bigtable row formats into narrow analytical rows (device_id, event_ts, metric, value, metadata JSON) or pre-computed aggregates if query patterns prefer it. Use Parquet/Avro intermediate files on GCS for large batch exports to reduce network overhead and enable retries without re-scanning Bigtable.

Operationally, build an incremental approach: for daily/ hourly loads keep a manifest of processed row-timestamp ranges or use changelog offsets if the ingestion system exposes them; for bulk backfills use snapshot-range exports. Add deduplication using message ids or last-write timestamps and ensure idempotent writes to BigQuery (MERGE by primary key or write to staging partitions then MERGE). Keep sample validation jobs that compare device-level counts and checksums between Bigtable extracts and BigQuery loads.

Security, cost and monitoring: run the pipeline in the same VPC/region to reduce egress, use service accounts with least privilege for Bigtable/BigQuery/GCS, and instrument metrics for rows processed, lag, and errors. Finally, document the contract (what's raw vs aggregated), maintain raw archives for reprocessing, and iterate on aggregation granularity after profiling real query patterns so the analytics tables remain fast and cost-efficient.

**49. PII fields must be masked before storage. How would you integrate Cloud DLP with Data Fusion?**

I'd treat Cloud DLP as the canonical de-identification engine and make masking a controlled, auditable step in the Data Fusion pipeline. Practically this means extracting the PII fields in a transform step, calling Cloud DLP's Deidentify API (or Inspect+Deidentify) to mask/tokenize/encrypt them, and then writing the de-identified result onwards. Because Data Fusion doesn't ship with a first-class DLP plugin out-of-the-box in many setups, I'd implement this via a lightweight custom transform plugin or by calling a Cloud Function / Dataflow template that wraps DLP; both patterns work for batch and streaming with slightly different latency trade-offs.

Decide the de-ident strategy based on use-case: deterministic tokenization or format-preserving encryption if you need consistent pseudonyms across records (use Cloud KMS-wrapped keys for re-identification control), or irreversible masking/hashing for analytics-only use. Use DLP templates to centralize rules (infoTypes, masking method, sampling), so pipelines remain consistent and compliant. For streaming low-latency paths, favor tokenization via a dedicated service with caching to reduce API calls; for batch jobs, call DLP in bulk mode or run a Dataflow job that groups records before calling DLP to control cost.

© Shubham Wagekar

Preserve provenance and governance: keep minimal metadata about what was masked (schema, template used, transformation version) in the output rows or an audit table, and route any records that DLP can't confidently process to a secure dead-letter store for manual review. Protect the key material and token maps - store any reversible token maps or re-ident tables in a locked, auditable location with strict IAM and encryption (Cloud KMS). Avoid storing plaintext PII unless you have a strong, documented reason and strict controls.

Operational concerns: rate limits and cost - Cloud DLP is not free and has throughput limits, so design batching, retries, backoff, and quotas into your Data Fusion transforms. Add monitoring on DLP API errors, volume of redacted fields, and latency. Include CI tests that assert masking behavior against sample PII and run privacy/regression checks when templates change.

Finally, document policy and runbooks: which fields are masked/mapped, re-identification approval flow (if allowed), and how to reprocess data if DLP templates change. That makes the masking predictable and auditable for auditors, product owners, and security teams.

**50. A dashboard must refresh using pipeline outputs. How would you integrate Looker Studio with Data Fusion?**

The simplest, most robust integration is to let Data Fusion write canonical analytics tables into BigQuery and connect Looker Studio directly to those BigQuery tables. Design the BigQuery tables with partitions, clustering, and pre-aggregated summary tables or materialized views so Looker Studio queries are fast and predictable. Keep stable table / view names for dashboards to avoid breaking data sources when you update pipelines.

Control freshness explicitly: if dashboards need near-real-time data, make the pipeline write streaming inserts or small micro-batches into partitioned BigQuery tables and configure Looker Studio's data source freshness/refresh interval accordingly. For heavier workloads or cost-sensitive environments, populate periodic aggregate tables (1-min/5-min/1-hour) and let Looker Studio refresh at a reasonable cadence - this often gives the best latency/cost balance.

Manage access and security by creating a service account or dataset-level "BI" user and grant it only the minimum BigQuery read access. Consider using authorized views in BigQuery to present curated subsets or to enforce row-level security without giving users direct access to raw tables. If Looker Studio caching or extracts are used, configure cache refresh schedules to match the pipeline's SLA so users see up-to-date data without excessive repeated query charges.

Performance and cost tuning: enable BI Engine for hot dashboards to accelerate repeated queries and reduce query costs; materialize frequently-used aggregates; and keep an eye on query cost with audit logs. Also set up monitoring/alerts for pipeline failures so dashboards don't silently display stale or partial data - a quick health metric (last successful pipeline run timestamp) shown on the dashboard helps users spot staleness.

Finally, operationalize the integration: keep pipeline and dashboard definitions in version control, document the data contract (fields, semantics, expected latency), and add a small validation step after each pipeline run that checks key metrics and updates a status table. That way dashboard owners can rely on a predictable, tested feed and engineers can quickly triage any data freshness or correctness issues.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

**51. A batch pipeline fails due to bad records. How would you capture and reprocess them?**

First I'd fail fast but not loudly: detect bad records early in the pipeline with a validation stage that checks schema, required fields, type conversions and business rules. Instead of letting the whole job crash, route offending rows to a dead-letter location (GCS or a "bad-rows" BigQuery table) with rich context - original payload, filename, line number (if file), pipeline id, timestamp, and a clear error code/reason.

Second, I'd keep the good rows flowing to the normal path so the business data isn't blocked by a few bad records. Record per-run metadata in a manifest table (files processed, row counts, bad row counts, checksums) so you can prove what was handled and why some rows went to DLQ. Emit metrics and alerts when bad-row rates exceed thresholds so the team treats it as an incident rather than background noise.

Third, create a reproducible reprocessing workflow: a small, idempotent repair job that reads from the DLQ, applies fixes (type coercion, trimming, enrichment, or manual corrections), and republishes to a reprocess topic or staging table. Make replays idempotent by using a unique processing key or writing to partitioned staging and then MERGE into the target with dedupe logic so you never double-insert.

Fourth, build operational tooling and runbooks: a simple UI or script to inspect sample bad records, mark which ones need manual fix vs automated fix, and kick off the reprocess. Keep provenance - who fixed what and why - in an audit table for compliance and to close the loop with the data producers so they can fix upstream problems.

Finally, add tests and prevention: add CI-driven contract checks against producer schemas, periodic sampling checks on incoming data to catch drift early, and keep the DLQ retention/archival policy so you can reprocess months-old data when needed. This makes bad-record handling observable, fixable, and safe to replay.

**52. A streaming pipeline drops failed messages. How would you design a dead-letter queue?**

Treat the DLQ as a first-class, observable stream. Emit failed messages to a dedicated Pub/Sub dead-letter topic (or a durable GCS/BigQuery archive for long-term storage) and include the full envelope: original message, subscription details, error type, pipeline version, timestamp, and attempt count. This metadata is crucial for triage and for safe replays.

Design retries upstream with exponential backoff and a capped retry budget; only after retries are exhausted route the message to the DLQ. For transient failures consider a retry topic that handles intermediate backoff before final DLQ, so the DLQ holds only truly poison or permanently malformed messages. Include an error category field so consumers can quickly filter by parsing errors vs enrichment/third-party failures.

For replayability, implement a small reprocess tool (or Cloud Function) that reads DLQ messages, optionally applies fixes or enrichment, and republishes to the original topic or a controlled reprocess topic. Ensure idempotency on replay by using message ids or a dedupe key so replays don't create duplicate rows in BigQuery or other sinks.

Operationalize: set retention and access controls on the DLQ, monitor DLQ growth and age (alerts if messages remain > SLA), surface top error reasons in a dashboard, and require an owner for DLQ spikes. Keep sample messages and summary metrics to speed up root-cause analysis and drive upstream fixes.

Finally, document the contract and policies: who can read/clear DLQ, how to replay safely, expected SLAs for triage, and how to escalate. With that, DLQs become a controlled recovery mechanism rather than a data graveyard.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

**53. A job fails midway - how would you configure retries in Data Fusion?**

I'd adopt multi-layer retries with safety and idempotency as first principles. At the orchestration level use the scheduler/orchestrator (Cloud Composer or Cloud Scheduler + retry logic) to automatically retry pipeline runs with exponential backoff and a sensible max-attempts count, so transient infra blips don't require manual intervention. For Dataproc-backed transforms, use job-level retry settings or orchestration retries that recreate ephemeral clusters to avoid stuck states.

At the pipeline level enable connector/plugin retries where available (source/sink connectors often have retry and backoff configs) so transient API/GCS/BigQuery failures are retried in-place. Complement this with circuit-breaker logic: if a repeated failure pattern emerges, fail fast to avoid thrashing and send alerts so engineers can investigate rather than wasting compute.

Make all retries safe by designing idempotent writes: use insertIds for streaming, MERGE/upsert semantics in BigQuery, or write to staging partitions and swap atomically. Track processing metadata (batch_id, file_name, generation/checksum) in a manifest so retries can detect already-processed inputs and skip duplicates.

Also add targeted retries for specific steps rather than blind full-pipeline reruns when possible: capture step-level failure state (intermediate GCS outputs, job counter) and allow retrying just the failed stage. Provide a manual reprocess playbook and a reusable replay tool that reads from manifests or DLQs and replays only what's needed.

Finally, monitor retry behavior (attempt counts, backoff intervals, time spent retrying) and surface it in alerts; if retries frequently mask an underlying bug, treat persistent retries as a signal to fix the root cause rather than increasing retry budgets. This keeps retries effective and the pipeline operable.

**54. You need to debug transformations that fail only on certain records. What would you do?**

I'd start by reproducing the failure with the smallest possible dataset - isolate a handful of failing records and the exact pipeline version that processed them. In an interview I'd say I always try to get a reproducible example quickly because debugging at scale is painful; spin up a local/dev run or a small ephemeral job that reads only the suspect records so I can iterate fast without waiting for full cluster runs.

Next, I'd add rich record-level logging and context: capture the raw input, the transformed intermediate values, the exact transform step, stack traces, and any schema/type coercions that occurred. I prefer structured logs that include pipeline_id, batch_id, record_id, and the error reason so I can pivot from logs to the specific record instantly instead of hunting through gigabytes of output.

I'd also add defensive validation earlier in the pipeline - assertions for required fields, type-checks, length limits, and sanity checks - that route bad rows to a dead-letter store instead of failing the whole job. That gives immediate visibility into the pattern of failures and preserves the bad payloads (with error metadata) so we can fix them or ask producers for changes without data loss.

Then I'd perform root-cause analysis: compare failing records to successful ones to find patterns (nulls, unexpected encodings, out-of-range values, key collisions, or skew). If the issue is flaky (depends on executor memory, parallelism, or timing), I'd reproduce with different parallelism and check for race conditions, non-deterministic UDFs, or reliance on unordered inputs.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

Finally, I'd fix and harden: patch the transform (safe casts, canonicalization, or fixing UDF determinism), add unit tests that include the failing examples, and add monitoring/alerts that detect regressions (spike in dead-letter rate or specific error codes). I'd also document the failure mode and add a replayable remediation flow so any archived bad records can be fixed and reprocessed safely.

**55. How would you design a fault-tolerant pipeline for critical financial transactions?**

I'd design for correctness first: require idempotent operations and end-to-end atomicity where possible. Practically that means each event carries a unique transaction id and the pipeline uses that id for deduplication and MERGE/upsert semantics at the sink so retries or duplicate deliveries never create incorrect balances or duplicate postings.

Second, build multiple durable layers: ingest via a reliable broker (Pub/Sub with message acks and ordering keys or another durable queue), persist raw events immediately to an immutable archive (GCS/Cloud Storage or a changelog table) for replay, and separate streaming processing from final commits so you always have the raw source to reconstruct state. This allows safe replays and forensic audits when things go wrong.

Third, enforce strict validation, sequencing, and compensation logic: validate schema and business rules early; keep per-account or per-aggregate ordering where required (ordering keys or partitioned processing); and implement compensating transactions or tombstones for reversals. For high-value transactions include multi-phase commits or a cleared-state table and mark transactions as pending/committed, so you never expose inconsistent interim states to downstream systems.

Fourth, harden operationally: use autoscaled, monitored compute (Dataflow or Dataproc with autoscaling), enforce retries with exponential backoff and capped attempts, and route unrecoverable messages to a secure DLQ for manual triage. Add end-to-end observability - per-transaction audit logs, lineage (which pipeline and transform version processed the record), SLA monitoring, and automated alerts for lag, error spikes, or duplicate detection.

Finally, treat security and compliance as essentials: encrypt data in transit and at rest, apply least-privilege IAM to pipeline components, keep detailed immutable audit trails for regulatory needs, and practice disaster recovery (cross-region backups, failover runs, and periodic reconciliation jobs that compare totals between source and sink). I'd finish by automating tests and runbooks - unit tests for transform correctness, chaos testing for failure modes, and clear operator playbooks for replay and rollback - so the pipeline is both correct and operable under pressure.

© Shubham Wadekar

**All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar**

**56. A junior developer needs access only to build pipelines but not deploy. How would you assign IAM roles?**

Give them the minimal things they need to author and test without giving deploy or production rights. Concretely, I'd create a scoped "pipeline-author" role that includes permissions to create, edit, and validate pipeline artifacts in the dev Data Fusion instance (or the repo where pipeline templates live), plus read access to related resources (schemas in BigQuery, sample data in GCS) so they can design and test locally. Don't grant any permissions that can change runtime infrastructure, such as creating clusters, modifying instance configs, or editing production instance settings.

Separate the "deploy" capability into a different role and workflow. Only CI/CD or a small set of release engineers/service accounts get the deploy role - this role has rights to run instances, create Dataproc clusters (or trigger Data Fusion runtime deploys), and invoke Cloud Build/Cloud Composer jobs. Enforce deploys through an automated pipeline (Cloud Build or equivalent) so human deploy rights are minimized and auditable.

Use IAM conditions and resource scoping to enforce environment boundaries: junior devs get the author role in the dev project/namespace only, not in staging/production. If your platform supports it, use namespace-level or folder-level bindings so the same identity can be an author in dev but not have any bindings in staging/prod. This prevents accidental cross-environment changes while letting them be productive.

Combine this with process controls: require code review and a CI pipeline that signs off schema changes, and require a pull-request + approved merge before any deployment job runs. Log all authoring and deploy actions and surface them in an audit dashboard so you can prove who changed what. Finally, document the exact permissions in an onboarding guide so juniors know how to request additional rights (and why the separation exists).

**57. Pipelines fail due to missing service account permissions. How would you fix this?**

First, diagnose precisely which permission is missing by inspecting the pipeline logs and the error messages - they usually indicate the API and permission that was denied (for example, BigQuery write, GCS read, Pub/Sub publish, or Secret Manager access). If logs aren't explicit, reproduce the failing step locally or in a dev run with the same service account to surface the exact IAM error. Never guess - grant only what is required.

Second, map the required actions to the smallest set of roles or permissions. Prefer assigning a narrowly-scoped predefined role (e.g., BigQuery Data Writer, Storage Object Viewer, Pub/Sub Publisher) or, better, a custom role that contains only the exact permissions the pipeline needs. Avoid blanket project-level Editor or Owner bindings. If the pipeline uses a particular sink/source, include the service account for that resource (e.g., BigQuery dataset-level IAM) rather than giving broad project access.

Third, apply changes in a controlled way: update IAM binding for the service account, document the change, and re-run the failing job. If the service account needs temporary elevated rights for a migration, use time-bound IAM conditions or short-lived tokens and then revoke them after the operation completes. Record the change in the change log so auditors can see who approved it.

Fourth, harden for the future: add automated tests in CI that run representative pipeline steps using the service account (smoke tests) to catch missing permissions before deployment. Also maintain a permissions manifest for each pipeline (which service account needs which dataset/topic/bucket permissions) and use it during onboarding and audits. If permissions errors are recurring, create a small "permission playbook" for common fixes so responders act quickly and consistently.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

**58. Auditors ask for data lineage. How would you provide this in Data Fusion?**

Treat lineage as both technical metadata and business documentation. Start by enabling and capturing pipeline metadata at design time: ensure each Data Fusion pipeline and each transform step is annotated with source table(s), target table(s), column mappings, and transformation intent (e.g., "derive customer_age from dob"). Those annotations are the raw material auditors need to connect sources to sinks and understand transformations at a column level.

Use the platform's metadata and lineage capabilities (or plug in a metadata/catalog product) to automatically record execution lineage: which pipeline version processed which runs, which input files or datasets contributed rows, the batch_id or window, and the exact output dataset/partitions. Persist execution metadata (run id, start/end times, input checksums, row counts, errors) in a lineage store so auditors can pull time-bound reports - not just static pipeline diagrams but concrete run-level provenance.

Integrate with your data catalog and governance tooling (Data Catalog, Glue-like registries, or a CMDB) so you can show end-to-end lineage across systems (GCS → Data Fusion → Dataproc → BigQuery). Export or surface column-level lineage (which input columns map to which output columns, with any transformations documented) and keep historical versions so auditors can see how lineage changed over time.

Provide runnable artifacts for auditors: dashboards or reports that show (a) the pipeline DAG and step descriptions, (b) run-level provenance for a given dataset/partition (input files, processing job, output table and partition), and (c) a sample trace for specific records if needed (record id → processing path). Back this with immutable logs (storage of raw inputs and execution metadata) so the lineage claims are auditable and replayable.

Finally, operationalize lineage generation: include lineage checks in CI (validate that new pipeline changes include updated column mappings), surface lineage metadata in runbook pages, and provide a simple "lineage export" process (CSV/JSON or catalog API) for auditors. This combination - design-time annotations, automatic run-time capture, data-catalog integration, and exportable evidence - satisfies auditors and keeps lineage useful for engineers too.

**59. Compliance requires metadata capture for all ingested data. How would you handle this?**

I'd design the pipeline so metadata capture is part of the ingestion process, not an afterthought. For every dataset/file/topic we ingest, I'd log metadata such as source system, file name, GCS bucket path, row count, checksum/CRC, schema version, ingestion timestamp, and the pipeline/job id that processed it. This metadata would be written into a dedicated "ingestion metadata" BigQuery table or a metadata store that auditors and engineers can query later.

At the pipeline level, I'd add a stage that collects and publishes this metadata after each run. In Data Fusion, this could be done with a custom plugin or by adding a sink to a BigQuery table specifically designed for lineage/metadata. For streaming, I'd capture per-batch offsets and commit timestamps; for batch, I'd record file lists and manifests. This makes it easy to prove which files were ingested, when, and into which partitions.

I'd also capture schema evolution metadata - store schema versions and differences between source and target so compliance teams can see how data shape changed over time. This can be logged alongside operational metrics (success/failure, retries, dead-letter counts) in the metadata store.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

Operationally, I'd automate this metadata capture so developers don't need to remember to add it every time. Provide a standard ingestion template with metadata logging built-in, and enforce checks in CI/CD to ensure pipelines write metadata before completion. Add monitoring that alerts if metadata entries are missing for a run - because if metadata isn't recorded, compliance is broken.

Finally, I'd integrate with a catalog or governance tool (like Data Catalog) so auditors can see metadata lineage across systems, and document the metadata model (what fields we capture, retention period, access rules). That way compliance gets consistent, queryable, and auditable metadata for every ingested dataset.

**60. You need CMEK-based encryption for sensitive data. How would you configure this in Data Fusion?**

I'd first identify which sinks/sources need CMEK - for example, BigQuery datasets, GCS buckets, or Pub/Sub topics. In GCP, CMEK means using a Cloud KMS key to encrypt resources instead of Google-managed keys. The pipeline itself doesn't encrypt/decrypt manually; rather, we configure the target services to use CMEK and ensure the pipeline's service account has access to the key.

For BigQuery, I'd create the dataset with CMEK enabled, specifying the KMS key when creating the dataset. Data Fusion pipelines writing into that dataset automatically inherit CMEK encryption, as long as the service account running the pipeline has the cloudkms.cryptoKeyEncrypterDecrypter role on the key. For GCS, I'd configure the bucket with CMEK by default (or specify it at the object write level) so all files are encrypted with that key.

In Data Fusion itself, when configuring profiles for Dataproc or other runtimes, I'd set CMEK in the profile so clusters provisioned by Data Fusion use CMEK for their disks. This ensures even intermediate data on Dataproc workers is encrypted with customer-managed keys. Similarly, if Pub/Sub is involved, I'd enable CMEK on the topic/subscription so messages are encrypted with the same KMS key.

Operationally, I'd manage CMEK keys carefully: enable automatic rotation in KMS, restrict IAM on keys to a small set of service accounts, and log all key usage for audits. If the pipeline service account doesn't have the right permissions, jobs will fail at runtime, so I'd test key access explicitly in lower environments before production.

Finally, I'd document the CMEK configuration per dataset/bucket/topic, keep keys regionally scoped to match data location, and align key policies with compliance needs (for example, disabling Google access to keys if regulations require). This way encryption is enforced end-to-end, transparent to the pipeline logic, and fully auditable for sensitive data.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

**61. A pipeline runs slowly due to sequential execution. How would you add parallelism?**

First, I'd profile the pipeline to find the true bottleneck - identify which steps are running sequentially, which ones are CPU- or I/O-bound, and where the wall-clock time is spent. In an interview I'd say you can't safely parallelize until you know whether the slow part is network, disk, a single-threaded transform, or simply poor partitioning.

Second, break the work into independent units that can run in parallel. For file-based or time-partitioned data, split by date or file shards and run multiple workers/tasks concurrently. For record-level transforms, increase parallelism by tuning the engine's parallelism settings (Spark partitions, number of Dataflow workers, or Data Fusion shuffle partitions) so transforms operate across more tasks instead of one serial executor.

Third, remove artificial serialization in the DAG: convert any sequential task chain that doesn't have dependencies into parallel tasks, introduce map-style transforms before reduce-like aggregations, and consider using fan-out/fan-in patterns - run many parallel transforms and then merge results with an efficient aggregation step.

Fourth, handle stateful operations carefully - windowed aggregations or joins sometimes require ordered processing; use partitioned keys (keyBy/deviceId/account) so state is local per key and can be processed concurrently. For skewed keys, apply salting or pre-aggregation to avoid a single hot partition dominating runtime.

Finally, test and tune: run a scaled POC, monitor task distribution, GC and shuffle metrics, and iterate on partition sizes and worker counts. Also ensure side-effects (writes to sinks, external APIs) are safe to run in parallel - use idempotent writes, batching, and connection pooling so parallelism actually reduces latency instead of creating new contention.

**62. A Dataproc-backed pipeline is underperforming. How would you resize clusters?**

I'd start by analyzing job metrics: look at executor utilization, CPU vs memory pressure, shuffle read/write sizes, GC pauses, and task failure patterns to determine whether we're CPU-starved, memory-starved, or IO-bound. This tells you whether to increase cores, memory per executor, or add more workers rather than blindly changing cluster size.

Next, pick the right worker mix: increase the number of workers to improve parallelism if tasks are CPU-bound and many slots are idle, or increase executor memory/cores if tasks frequently OOM or spend time in GC. Consider using more smaller workers (horizontal scale) for many small tasks, or fewer larger workers (vertical scale) for memory-heavy jobs. Tune spark.executor.cores and spark.executor.memory to get a good per-executor balance and leave room for YARN/container overhead.

Third, leverage autoscaling and preemptible workers to balance cost and performance: enable autoscaling policies so the cluster grows for heavy stages and shrinks when idle; use preemptible workers for non-critical capacity with enough non-preemptible cores to handle the control plane. Also consider using local SSDs on workers temporarily if shuffle I/O is the bottleneck.

Fourth, reduce overhead before adding hardware: optimize shuffle partitions (spark.sql.shuffle.partitions), compact small files, convert data to Parquet, and push down predicates so you read less. Often performance gains come from reducing shuffle and I/O, not just adding nodes.

Finally, validate changes with representative runs: adjust cluster templates as parameters, run a few sized experiments, and pick the smallest configuration that meets SLA. Document the sizing rationale so future runs use the tuned template and avoid overprovisioning while keeping predictable performance.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

**63. Your ingestion pipeline creates too many small files in GCS. How would you optimize this?**

First, stop the root cause: identify why many small files are produced - per-record writes, high parallelism with tiny partitions, or many micro-batches. Change the producer to write larger bundles (buffer and flush after N records or size threshold) or write temporary files and atomically move/rename completed files so consumers see only finalized larger files.

Second, use write-side batching/compaction in the pipeline: buffer records in memory or local temp files and write larger files (aim for 100MB–1GB depending on downstream readers). If you're using Spark/Dataproc/Dataflow, tune the number of output partitions or coalesce/repartition before the sink to reduce file count and control file sizes.

Third, run a post-write compaction job if producers can't be changed: periodically compact small files into larger Parquet/Avro files using a scheduled Dataflow/Data Fusion/Dataproc job that reads the small objects and writes consolidated files, then update the manifest/markers so consumers use the compacted set.

Fourth, choose columnar formats and tune file-size targets: writing Parquet with Snappy and reasonable row-group sizes gives better compression and read performance. Also write partitioned directories (year=, month=, day=) so you keep discoverability while limiting small-file explosion in each partition.

Finally, add operational guards: implement a manifest with file counts and average file size metrics, alert if files-per-minute exceed thresholds, and include compaction as part of your SLA-runbook. That makes the ingestion sustainable for downstream analytics and avoids the common small-file performance pain.

**64. A poorly designed schema is slowing down throughput. How would you fix it?**

First, I'd profile the workload and queries to identify the real pain points: which queries are slow, which joins or scans are heavy, and whether the bottleneck is IO, CPU, or shuffle. You can't fix schema problems blind - start with metrics (query time, bytes read, hot partitions) so changes target the biggest wins.

Second, normalize versus denormalize based on access patterns. For OLAP workloads, denormalize and store pre-joined or flattened rows (or materialized views) to avoid expensive joins. For OLTP-like access or when writes dominate, normalize to avoid update anomalies. Let query patterns drive the shape, not theoretical purity.

Third, redesign partitions and clustering. Partition the table by a high-cardinality time or business column used in filters, and add clustering (or secondary indexes where available) on frequently filtered or join keys. Good partitioning reduces scanned bytes drastically and clustering groups related rows to speed lookups and joins.

Fourth, fix data types and nullability: use the narrowest sensible types, avoid wide VARCHARs when INTEGERs suffice, and make columns nullable only when necessary. Also convert wide JSON blobs into typed columns or keep them in a JSON/STRUCT extension column - typed columns are cheaper to scan and easier to optimize.

Fifth, introduce pre-aggregation and compact storage formats: roll up high-cardinality telemetry into summary tables for common queries, write columnar formats (Parquet/ORC) for large raw stores, and compact small files. Finally, iterate - run benchmarks after each change, keep a migration plan (backfill strategy, compatibility checks), and document the new schema and why it's faster so future engineers don't accidentally revert the improvements.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

**65. A stakeholder asks why batch is cheaper than streaming in some cases. How would you explain this?**

I'd start by saying the core reason is amortization: batch groups many records into fewer, larger operations, so fixed costs (RPCs, job startup, metadata overhead) are paid once per batch rather than per record. Streaming usually pays per-message or for continuous resources, so small, frequent events add up to higher per-record cost.

Second, streaming often forces lower-latency infrastructure and different consistency mechanisms (continuous worker instances, frequent small inserts, at-least-once semantics with dedupe), which increases compute and operational overhead. Batch can use short-lived, highly optimized jobs and efficient bulk-load paths (load jobs into warehouses) that are cheaper at scale.

Third, storage and query cost differences matter: batch lets you compact and compress more effectively (larger Parquet files, coalesced partitions), reducing storage I/O and downstream query bytes. Streaming tends to create lots of small objects or streaming inserts that are less efficient for columnar stores and may incur higher query or ingestion charges.

Fourth, complexity and reliability add hidden cost: streaming systems need more monitoring, backpressure handling, and DLQ workflows, which increases engineering effort and operational burn. Batch can often be simpler to test, replay, and reason about, lowering ongoing maintenance cost - that's a real component of "cheaper" over time.

Finally, it's not universally true - if you need true sub-second visibility or the business values immediacy highly, streaming's higher cost is justified. I'd recommend quantifying the latency requirement, running a small cost/latency POC for both approaches, and choosing the one that meets business SLAs at the best total cost of ownership.

**66. You must design a CDC pipeline from Cloud SQL to BigQuery. How would you build this?**

I'd start with a CDC capture service - use Datastream (or a managed CDC tool) to capture the initial snapshot and the ongoing change events from Cloud SQL and publish them into Pub/Sub (or write to GCS as a durable change log). That gives a reliable, ordered changelog without hammering the source DB and provides a consistent starting point for replays and audits.

Next, process the change events with a streaming job (Dataflow). The job parses the CDC envelope, normalizes types, filters or enriches as needed, and converts events into idempotent operations (upsert/delete) that BigQuery can consume. Include the primary key, operation type, commit timestamp/LSN and a source offset in each envelope so downstream merges are deterministic and replays are safe.

For applying changes to BigQuery, stream into a staging table (or use the BigQuery Storage Write API with insertIds) and run near-real-time MERGE statements that upsert by primary key using commit timestamps to pick the latest change. This staging+MERGE pattern gives correctness (handles updates/deletes) and idempotency for retries and replays, avoiding duplicate or out-of-order effects in the production table.

Operationally, add schema-evolution handling (store unknown fields in an extensions JSON or alert for schema drift), dead-letter handling for malformed events, and monitoring for replication lag, Pub/Sub backlog, and merge failures. Keep raw change events archived in GCS for forensic replays and create a manifest/offset store so you can resume from the last consistent point if the pipeline needs recovery.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

Finally, lock down security and test thoroughly: use VPC/private connectivity from Datastream to Cloud SQL, give the pipeline service accounts least privilege (BigQuery Data Editor, Pub/Sub Subscriber, KMS perms if using CMEK), and run a shadow/parallel run to validate parity before cutting over. Document runbooks for replay, backfill, and reconciliations (row counts, checksums) so you can prove correctness to stakeholders.

**67. Your company wants to stream website clickstream data into BigQuery in near real time. How would you design the pipeline?**

My default pattern is Pub/Sub → Dataflow → BigQuery. Producers (web clients or edge collectors) push events into Pub/Sub; Dataflow consumes the topic, does lightweight validation, sessionization/enrichment (IP→geo, user-agent parsing) and windowing if needed, and writes to BigQuery using streaming inserts or the Storage Write API for low-latency ingestion. This gives sub-second to second visibility while keeping the pipeline scalable and manageable.

Keep messages compact and versioned: serialize as JSON/Avro with a schema_version field and include an event_id and timestamp to support dedupe and ordering. In the transform layer, validate schema and route bad events to a dead-letter topic/GCS for investigation. Use insertIds or an idempotent MERGE pattern at the sink so retries and at-least-once delivery won't cause duplicates in analytics tables.

Design BigQuery tables for analytics: partition by event_date (ingestion date) and cluster by user_id/session_id or page to accelerate common queries. Precompute common aggregates (per-minute counts, session summaries) into separate summary tables if dashboards require very low-latency metrics - this reduces repeated scanning of raw click data and controls cost.

Operational concerns matter: autoscale Dataflow workers, monitor Pub/Sub subscription lag and Dataflow processing latency, alert on DLQ spikes, and keep a raw archive of events in GCS (partitioned by date) for replay/backfills. If ordering per-user matters, use Pub/Sub ordering keys or key-by-user in Dataflow to keep per-user ordering while still parallelizing across users.

Finally, plan for cost and evolution: measure ingestion and query costs, consider micro-batching (1–30s) if you can trade a little latency for lower cost, and maintain a schema-evolution policy so frontend changes don't break pipeline consumers. Version transforms, run compatibility tests, and document the event contract so product and analytics teams can evolve safely.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

**68. You need to ingest large historical files from GCS into BigQuery. How would you implement this?**

For large historical loads, prefer bulk load jobs over streaming. Stage files in an organized GCS layout (prefer columnar formats like Parquet/Avro or newline-delimited JSON/CSV if necessary), validate a few sample files, and then use BigQuery load jobs (or the bq load API) to efficiently import whole files into partitioned staging tables. Load jobs are cheaper and faster for large volumes and handle schema inference/casting robustly.

Make the process idempotent and track progress with a manifest table: record file name, checksum/generation, start/end times, row counts, and load status. The pipeline should skip files already marked as successfully loaded and report failures with rich error context. This manifest enables safe retries and avoids duplicate ingestion when jobs are re-run.

After loading into staging, run validation and transformation SQL in BigQuery to map staging columns to production schema, handle type conversions, and deduplicate (MERGE into the final production table by primary key or business key). Doing transformations in BigQuery for large historical data leverages its bulk-processing efficiency and avoids moving data unnecessarily.

If files are too large to load in one run or need pre-processing (parsing, enrichment, or splitting), use a Dataflow/Data Fusion job to read from GCS, transform, write out optimized Parquet partitions, and then issue BigQuery load jobs. For very large backfills, run in parallel by partition (date ranges) to speed throughput while respecting BigQuery quotas and project quotas.

Finally, validate completeness and parity: compare source file row counts and checksums with loaded rows, run sample queries to ensure expected aggregates match, and keep an archival copy of original files and the manifest for audit. Schedule a controlled cutover and communicate with stakeholders about availability and any downstream data contract changes so consumers can rely on the historical dataset.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

**69. A pipeline must validate, cleanse, and enrich data before loading. How would you design it?**

I'd start by treating validation as the first, non-negotiable step: validate schema, required fields, types, and basic business rules as soon as data enters the pipeline. In practice I build a small "ingest validation" stage that checks message/file-level integrity (checksums, completeness) and row-level assertions (non-null keys, timestamp sanity). Failures go to a dead-letter store with rich context so good data can keep flowing while bad records are quarantined for review.

Next, cleansing is explicit and idempotent: canonicalize strings (trim/case/normalize), safely cast types with fallbacks, remove or flag outliers, and standardize formats like timestamps and IDs. I implement these as deterministic transforms so reprocessing yields the same result; every transform step emits metrics (counts, percent fixed, examples) so we can measure data health trends and catch regressions early.

For enrichment I prefer a layered approach: lightweight, cached enrichments (geo-IP, user agent parsing, static lookups) happen inline in the streaming path when latency matters; heavier joins or reference-data lookups (CRM enrichment, complex entity resolution) run in a parallel async stage or a downstream micro-batch. This keeps low-latency flows fast while still delivering rich records to analytics after a short delay if needed.

Operationally, I'd write cleansed output to a staging area (partitioned BigQuery table or GCS parquet) and run a controlled MERGE into production using primary-key upserts. That gives clear checkpoints, makes retries safe, and separates transformation logic from final commit semantics. It also makes backfills and audits straightforward because the staging data is the single source of truth for each run.

Finally, governance and observability are baked in: schema-contract tests in CI, automated schema-drift alerts, a manifest table that records file-level metadata and processing results, and a small validation job that compares source vs landed counts and key aggregates. Document the rules and maintain a runbook for reprocessing DLQ items so the team can quickly triage and remediate issues without risking data correctness.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

**70. Your company wants to migrate Informatica ETL jobs to GCP. How would you use Data Fusion?**

I'd begin with an inventory and classification of Informatica jobs: categorize by complexity (simple mappings, parameterized flows, heavy transformations, custom Java), data sources/sinks, SLAs, and dependencies. That inventory tells us which jobs map cleanly to Data Fusion pipelines versus which require a Dataproc/Spark or Dataflow reimplementation for performance or custom-code reasons.

Next, for straightforward mappings and connectors, I'd re-create the flows in Data Fusion using its visual pipeline canvas and native connectors (GCS, BigQuery, Pub/Sub, JDBC). Where Informatica used built-in transforms, map them to Data Fusion transforms one-to-one when possible; for complex logic, encapsulate it in a Dataproc/Spark step or a reusable plugin so we keep behavior consistent and testable.

I'd run a phased migration: start with low-risk jobs to build confidence (non-critical reporting ETL), run in shadow mode alongside Informatica for parity checks, and validate outputs with automated tests (row counts, checksums, business-key aggregates). Use a CI/CD pipeline to version Data Fusion artifacts and enforce contract tests so the migration is repeatable and auditable.

Operationally, integrate with Dataproc for heavy jobs (use a Dataproc profile from Data Fusion), implement manifests and fail-safe staging tables, and adopt the same monitoring/alerting standards as on-prem. Also design a rollback/replay strategy: keep raw archives in GCS and use the manifest to replay jobs into BigQuery if issues appear after cutover.

Finally, involve stakeholders early — document functional equivalence, list any expected changes in SLAs or latency, and provide runbooks and training for operators. The goal is not just a lift-and-shift but a pragmatic re-platform: preserve behavior where necessary, optimize where possible, and hand over a maintainable, cloud-native ETL surface that the team can own going forward.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar